Learning Data-adaptive Nonparametric Kernels

Fanghui Liu, Xiaolin Huang, *Senior Member, IEEE*, Chen Gong, *Member, IEEE*,
Jie Yang, and Li Li, *Fellow, IEEE*

Abstract—Kernel methods have been extensively used in a variety of machine learning tasks such as classification, clustering, and dimensionality reduction. For complicated practical tasks, the traditional kernels, *e.g.*, Gaussian kernel and sigmoid kernel, or their combinations are often not sufficiently flexible to fit the data. In this paper, we present a **D**ata-**A**daptive **N**onparametric **K**ernel (DANK) learning framework in a data-driven manner. To be specific, in model formulation, we impose an adaptive matrix on the kernel/Gram matrix in an entry-wise strategy. Since we do not specify the formulation of the adaptive matrix, each entry in the adaptive matrix can be directly and flexibly learned from the data. Therefore, the solution space of the learned kernel is largely expanded, which makes our DANK model flexible to capture the data with different local statistical properties. Specifically, the proposed kernel learning framework can be seamlessly embedded to support vector machines (SVM) and support vector regression (SVR), which has the capability of enlarging the margin between classes and reducing the model generalization error. Theoretically, we demonstrate that the objective function of our DANK model embedded in SVM/SVR is gradient-Lipschitz continuous. Thereby, the training process for kernel and parameter learning in SVM/SVR can be efficiently optimized in a unified framework. Further, to address the scalability issue in nonparametric kernel learning framework, we decompose the entire optimization problem in DANK into several smaller easy-to-solve problems, so that our DANK model can be efficiently approximated by this partition. The effectiveness of this approximation is demonstrated by both empirical studies and theoretical guarantees. Experimentally, the proposed DANK model embedded in SVM/SVR achieves encouraging performance on various classification and regression benchmark datasets when compared with other representative kernel learning based algorithms.

Index Terms—adaptive matrix, SVM, nonparametric kernel learning

arXiv:1808.10724v1 [cs.LG] 31 Aug 2018

1 Introduction

Kernel methods [1], [2] such as support vector machines (SVM) [3], support vector regression (SVR) [4], and Gaussian process regression [5] have been shown effective in a variety of machine learning tasks. The rationale behind these kernel methods relies on an implicit mapping from the original input space to the (high-dimensional or infinite-dimensional) feature space. In practical use, there is no need to acquire the explicit formulation of such mapping. Instead, what we should obtain is only the inner product of two vectors in the feature space by introducing a *kernel*, which is the well-known "kernel trick".

Generally, the performance of kernel methods largely depends on the choice of the kernel. Traditional kernel methods often adopt a classical kernel, *e.g.*, Gaussian kernel or sigmoid kernel for characterizing the relationship among data points. Empirical studies suggest that these traditional kernels are not sufficiently flexible to depict the domain-specific characteristics of data affinities or relationships. To address such limitation, several routes have been explored. One way is to design sophisticated kernels on specific tasks such as applying optimal assignment kernels [6] to graph classification, developing a kernel based on triplet comparisons [7], or even breaking the restrict of positive definiteness on the traditional kernel [8], [9]. Apart from these well-designed kernels, a bunch of research studies aim to automatically learn effective and flexible kernels from data, known as *learning kernels*. Algorithms for learning kernels can be roughly grouped into two categories: parametric kernel learning and non-parametric kernel learning.

F. Liu, X. Huang and J. Yang are with Institute of Image Processing and Pattern Recognition, Shanghai Jiao Tong University, Shanghai 200240, China (e-mail: {lfhsgre;xiaolinhuang;jieyang}@sjtu.edu.cn).
C. Gong is with the Key Laboratory of Intelligent Perception and Systems for High-Dimensional Information of Ministry of Education, School of Computer Science and Engineering, Nanjing University of Science and Technology, Nanjing, 210094, P. R. China (email: chen.gong@njust.edu.cn).
L. Li is with the Department of Automation, Tsinghua University (email: li-li@mail.tsinghua.edu.cn).
Corresponding authors: Jie Yang and Xiaolin Huang.

1.1 Review of Kernel Learning

Let $\mathcal{D} = \{(\boldsymbol{x}_i, y_i)\}_{i=1}^n$ be the training set with n training examples, where $\boldsymbol{x}_i \in \mathcal{X} \subseteq \mathbb{R}^d$ and $y_i \in \mathcal{Y}$. Let $k(\cdot,\cdot)$ be a positive definite kernel function and $\boldsymbol{K} = [k(\boldsymbol{x}_i, \boldsymbol{x}_j)]_{n\times n}$ be the kernel/Gram matrix computed from $\mathcal{D}$.

In parametric kernel learning, the learned kernel $k(\cdot,\cdot)$ is assumed to have a specific parametric form, of which the optimal parameters are learned on the given data. A representative approach is multiple kernel learning (MKL) [10], which aims to learn a linear combination of a predefined parametric kernel set with base kernels $\{k_t\}_{t=1}^s$, namely

$$k(\boldsymbol{x}_i, \boldsymbol{x}_j) = \sum_{t=1}^{s} \mu_t k_t(\boldsymbol{x}_i, \boldsymbol{x}_j)\,,$$

where $\boldsymbol{\mu} = [\mu_1, \mu_2, \ldots, \mu_s]^\top$ is the weight vector that can be restricted by the conic sum (*i.e.*, $\mu_t \geq 0$), the convex sum (*i.e.*, $\mu_t \geq 0$ and $\sum_{t=1}^s \mu_t = 1$), or various regularizers such as ℓ_1 norm, mixed norm, and entropy-based formulation (see a survey in [11], [12]). By doing so, MKL would generate a "broader" kernel to enhance the representation ability for data. Besides, hierarchical kernel learning (HKL) [13] and spectral mixture models [14], [15] are also two typical techniques in parametric kernel learning for discovering flexible statistical representations in data.

Compared to parametric kernel learning, the algorithms belonging to nonparametric kernel learning are often more flexible as they directly learn a positive semi-definite (PSD) kernel matrix in a data-specific manner. For instance, a target label kernel (matrix) is defined by $\boldsymbol{K} = \boldsymbol{y}\boldsymbol{y}^\top$ with the label vector $\boldsymbol{y}$. It is an ideal kernel that directly recognises the training data with certainly 100%

accuracy, and thus can be used to guide the kernel learning task. Kernel target alignment [16] is a typical method that measures the similarity between the learned Gram matrix and the ideal kernel. Such technique is subsequently incorporated into [17] for kernel learning through the optimized random features. Another line of nonparametric kernel learning attempts to directly learn the values in $\boldsymbol{K}$ [18], [19], [20]. The earliest work is proposed by Jordan et al. [18], in which they aim to simultaneously optimize the kernel matrix $\boldsymbol{K}$ restricted in a solution space $\mathcal{K}$ and the parameters in SVM classifier. The formulation of this nonparametric kernel learning framework embedded in SVM is

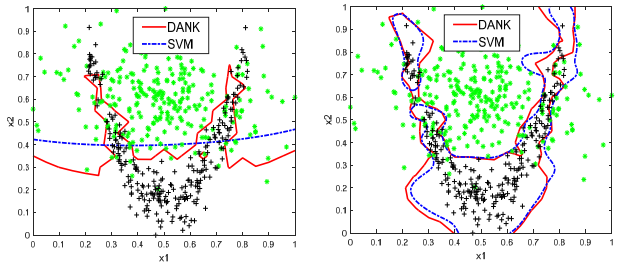
$$\min_{\boldsymbol{K}\in\mathcal{K}} \max_{\boldsymbol{\alpha}} \ \mathbf{1}^\top\boldsymbol{\alpha} - \frac{1}{2}\boldsymbol{\alpha}^\top \operatorname{diag}(\boldsymbol{y})\boldsymbol{K}\operatorname{diag}(\boldsymbol{y})\boldsymbol{\alpha}$$
$$s.t.\ \ \boldsymbol{\alpha}^\top\boldsymbol{y} = 0,\ \mathbf{0} \le \boldsymbol{\alpha} \le C\mathbf{1},\ tr(\boldsymbol{K}) \le c\,,$$

where $\boldsymbol{\alpha} \in \mathbb{R}^n$ is the dual variable, $\mathbf{1}$ is the all-one vector, C is the balance parameter in SVM, and c controls the complexity of $\boldsymbol{K}$. In [18], the solution space is assumed to be $\mathcal{K} = \{\sum_{i=1}^t \mu_i \boldsymbol{K}_i : 0 \le \mu_i \le 1, i = 1, 2, \ldots, t\} \cap \{\boldsymbol{K} \in \mathcal{S}_+^n\}$, which lies in the intersection of a low-dimensional linear subspace and the positive semi-definite cone $\mathcal{S}_+^n$. In this case, optimizing $\boldsymbol{K}$ is transformed to optimize the weight vector $\boldsymbol{\mu}$ by semi-definite programming. Such kernel learning framework is then applied to geometry-aware kernel learning [21], [22] by fully exploiting all known geometry information about the given data.

1.2 Contributions

From the above discussions, we observe that, although most nonparametric kernel learning works recognize the significance of directly learning kernel values, they still assume that each entry in the Gram matrix $\boldsymbol{K}$ is learned by a combination of the values in predefined base kernel matrices. For example, in [16], [18], each entry in $\boldsymbol{K}$ is learned from a linear combination of some "guess" base kernel matrices. In [23], base kernel matrices are constructed by principal eigenvectors of the observed data. These methods are still halfway in nonparametric kernel (value) learning. The model flexibility is significantly limited to the formulation of predefined base kernel matrices, as they may be not rich enough to contain the optimal kernel to fit the data.

To address this limitation, we propose a **D**ata-**A**daptive **N**onparametric **K**ernel (DANK) learning framework, in which a data adaptive matrix is imposed on the pre-given kernel matrix. Since we do not specify the formulation of the adaptive matrix $\boldsymbol{F}$, each entry in $\boldsymbol{F}$ can be directly and flexibly learned from the data. Such formulation-free strategy yields a quite flexible nonparametric kernel matrix $\boldsymbol{F} \odot \boldsymbol{K}$ where $\odot$ denotes the Hadamard product between two matrices. As a result, our DANK model provides adequate model flexibility to capture the data with different local statistical properties. Here we take a synthetic classification dataset *clowns* to illustrate its strong data adaptivity. The initial kernel matrix $\boldsymbol{K}$ is given by the classical Gaussian kernel $k(\boldsymbol{x}_i, \boldsymbol{x}_j) = \exp(-\|\boldsymbol{x}_i - \boldsymbol{x}_j\|^2/\sigma^2)$ with the width σ. Fig. 1 shows that, in the left panel, the baseline SVM with an improper $\sigma = 1$ cannot accurately adapt to the data. The generated classification boundary is inaccurate, resulting in unsatisfactory classification performance. However, based on the same $\boldsymbol{K}$, by optimizing the adaptive matrix $\boldsymbol{F}$, our DANK model shows good flexibility to fit the complex data distribution, leading to a desirable decision boundary. Comparably, in the right panel, even under the condition that σ is well tuned via cross validation, by learning $\boldsymbol{F}$,



(a) $\sigma = 1$ (b) $\sigma = 0.1$ by cross validation

Fig. 1. Classification boundaries of SVM (in blue dash line) and our DANK model (in red solid line): (a) using the Gaussian kernel with $\sigma = 1$; (b) using the Gaussian kernel with $\sigma = 0.1$ by cross validation.

our DANK model still yields an more accurate boundary than SVM to fit the data points.

The main contributions of this paper lie in the following three folds: 1) We propose a data-adaptive nonparametric kernel learning model termed "DANK" to enhance the model flexibility and data adaptivity of traditional nonparametric kernel learning framework, which can be seamlessly embedded to SVM and SVR for classification and regression tasks; 2) The DANK model and the induced classification/regression model can be formulated as a max-min optimization problem in a unified framework, of which the objective function is gradient-Lipschitz continuous, and can be directly solved by a projected gradient method with Nesterov's acceleration; 3) By decomposing the entire optimization problem into several smaller easy-to-solve subproblems, we propose an accelerated strategy to handle the scalability issue that appears in our nonparametric kernel learning framework. The effectiveness of our decomposition-based scalable approach is demonstrated by both theoretical and empirical studies. The experimental results on several classification and regression benchmark datasets demonstrate the superiority of the proposed DANK framework over other representative kernel based methods.

This paper is an extension of our previous conference work [24]. The improvements of this paper are mainly in the following four aspects. First, in model formulation, we impose an additional low rank constraint and a bounded regularizer on $\boldsymbol{F}$, which makes the original model easy to deal with out-of-sample extensions problem. Apart from SVM to which our DANK model is embedded for classification, the proposed DANK model is also extended to SVR for regression tasks. Second, in algorithmic aspect, we develop a Nesterov's smooth method to directly solve the designed optimization problem. Third, in large-scale situations, we conduct a decomposition-based scalable approach on DANK with theoretical guarantees and experimental validation. Lastly, we provide more experimental results on popular benchmarks.

The remainder of the paper is organized as follows. Section 2 introduces the proposed DANK model embedded in SVM, of which the model optimization is presented in Section 3. In Section 4, we extend our DANK model to SVR for regression. Kernel approximation on large-scale datasets is explained in Section 5. The experimental results on popular benchmark datasets are presented in Section 6. Section 7 concludes the entire paper.

2 The DANK Model in SVM

In this section, we briefly review the SVM formulation and then incorporate the proposed DANK model into SVM for classification.

We begin with the hard-margin SVM for linearly separable classification tasks. It aims to learn a linear classifier $f(\boldsymbol{x};\boldsymbol{w},b) = \text{sign}(\boldsymbol{w}^\top\boldsymbol{x}+b) \in \{-1,+1\}$ with $\boldsymbol{w}$ and b that determine the decision hyperplane. The underlying idea of SVM is to find the hyperplane $(\boldsymbol{w},b)$ which yields the maximal distance between the nearest training examples of the two classes (a.k.a the *margin* $\gamma = 1/\|\boldsymbol{w}\|$), as this way reduces the model generalization error [3]. While the data are not linearly separable in most practical settings, the hard-margin SVM is subsequently extended to soft-margin SVM with an implicit mapping $\phi(\cdot)$ for generating a non-linear decision hyperplane. Mathematically, the soft-margin SVM aims to maximize the margin γ (or minimize $\|\boldsymbol{w}\|^2$) and minimize the slack penalty $\sum_{i=1}^n \xi_i$ with the following formulation

$$\begin{aligned} &\min_{\boldsymbol{w},b,\boldsymbol{\xi}} \frac{1}{2}\|\boldsymbol{w}\|^2 + C\sum_{i=1}^n \xi_i \\ &\text{s.t. } y_i(\boldsymbol{w}^\top\phi(\boldsymbol{x}_i)+b) \geq 1-\xi_i,\ \xi_i \geq 0,\ i=1,2,\cdots,n\,, \end{aligned} \tag{1}$$

where $\boldsymbol{\xi} = [\xi_1,\xi_2,\cdots,\xi_n]^\top$ is the slack variable, and C is the balance parameter. As mentioned in [3], the dual form of problem (1) is given by

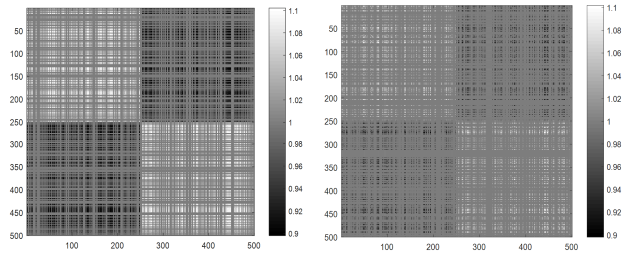
$$\begin{aligned} &\max_{\boldsymbol{\alpha}}\ \mathbf{1}^\top\boldsymbol{\alpha} - \frac{1}{2}\boldsymbol{\alpha}^\top\mathbf{Y}\mathbf{K}\mathbf{Y}\boldsymbol{\alpha} \\ &\text{s.t. } 0 \leq \boldsymbol{\alpha} \leq C\mathbf{1},\ \boldsymbol{\alpha}^\top\mathbf{y} = 0\,, \end{aligned} \tag{2}$$

where $\boldsymbol{Y} = \text{diag}(\boldsymbol{y})$ is the label matrix and $\boldsymbol{K} = [k(\boldsymbol{x}_i,\boldsymbol{x}_j)]_{n\times n}$ is the Gram matrix, satisfying $k(\boldsymbol{x}_i,\boldsymbol{x}_j) = \langle\phi(\boldsymbol{x}_i),\phi(\boldsymbol{x}_j)\rangle$. Since problem (2) is convex, strong duality holds by Slater's condition [25]. The optimal values of the primal and dual soft-margin SVM problems will be equal. Accordingly, by learning a flexible kernel matrix in the dual form of SVM in problem (2), the objective function value of the primal problem in problem (1) would decrease, which in turn increases the margin γ. Theoretically, with probability at least $1-\delta$, the estimation error (namely the gap between empirical error and expected error) of a learned SVM classifier with a larger margin γ achieves a tighter bound $\sqrt{\mathcal{O}(1/\gamma^2 - \log\delta)/n}$ [26]. Based on above analyses, we conduct the nonparametric kernel learning task in our DANK model by introducing an adaptive matrix $\boldsymbol{F}$ into problem (2), that is

$$\min_{\boldsymbol{F}\in\mathcal{F}}\ \max_{\boldsymbol{\alpha}\in\mathcal{A}}\ \mathbf{1}^\top\boldsymbol{\alpha} - \frac{1}{2}\boldsymbol{\alpha}^\top\boldsymbol{Y}\big(\boldsymbol{F}\odot\boldsymbol{K}\big)\boldsymbol{Y}\boldsymbol{\alpha}\,, \tag{3}$$

where $\mathcal{F}$ is the feasible region of $\boldsymbol{F}$, and the constraint for the standard SVM is expressed as $\mathcal{A} = \{\boldsymbol{\alpha}\in\mathbb{R}^n : \boldsymbol{\alpha}^\top\boldsymbol{y} = 0,\ 0 \leq \boldsymbol{\alpha} \leq C\mathbf{1}\}$. Optimizing $\boldsymbol{F}$ yields a flexible kernel matrix $\boldsymbol{F}\odot\boldsymbol{K}$, which is able to increase the margin, and further allows for greater model flexibility with a tighter estimation error as well as a lower model generalization error when compared to the original SVM classifier.

In our DANK model, since we do not specify the formulation of $\boldsymbol{F}$, the solution space $\mathcal{F}$ is enlarged to boost the model flexibility and capacity of fitting diverse data patterns. However, arbitrarily enlarging the solution space $\mathcal{F}$ could bring about significant difficulty in out-of-sample extensions [27], [28], [29]. That is, the flexible kernel matrix $\boldsymbol{F}\odot\boldsymbol{K}$ is learned from the training data in a nonparametric manner, while the adaptive kernel matrix for test data is unknown. Such out-of-sample extension problem is a common issue in nonparametric kernel learning. To alleviate this problem, the learned $\boldsymbol{F}$ is expected to vary steadily and smoothly between any two neighboring data points in a suitable space, and thus can be easily extended to the adaptive matrix $\boldsymbol{F}'$ for test data.



(a) (b)

Fig. 2. The range of $\boldsymbol{F}$ in our DANK model with the Gaussian kernel: (a) $\sigma = 1$; (b) $\sigma = 0.1$ by cross validation.

Intuitively speaking, when $\boldsymbol{F}$ is chosen as the all-one matrix, the DANK model in Eq. (3) degenerates to a standard SVM problem. Such all-one matrix associated with its rank-one property guides us to design the suitable space $\mathcal{F}$ in two aspects. First, the adaptive matrix $\boldsymbol{F}$ is expected to vary around the all-one matrix in a bounded region for a trade-off between its complexity and flexibility. Second, $\boldsymbol{F}$ is desired to be endowed with the low rank structure enjoyed by the all-one matrix. Mathematically, our DANK model is

$$\begin{aligned} &\min_{\boldsymbol{F}\in\mathcal{S}_+^n}\ \max_{\boldsymbol{\alpha}\in\mathcal{A}}\ \mathbf{1}^\top\boldsymbol{\alpha} - \frac{1}{2}\boldsymbol{\alpha}^\top\boldsymbol{Y}\big(\boldsymbol{F}\odot\boldsymbol{K}\big)\boldsymbol{Y}\boldsymbol{\alpha} \\ &\text{s.t. } \|\boldsymbol{F}-\mathbf{1}\mathbf{1}^\top\|_{\text{F}}^2 \leq R^2,\ \text{rank}(\boldsymbol{F}) < r\,, \end{aligned} \tag{4}$$

where R refers to the bounded region size, $\text{rank}(\boldsymbol{F})$ denotes the rank of $\boldsymbol{F}$, and $r \leq n$ is a given integer. In our DANK model, $\boldsymbol{F}\in\mathcal{S}_+^n$ is given to ensure that the learned kernel matrix $\boldsymbol{F}\odot\boldsymbol{K}$ is still a PSD one[1]. The bounded constraint also prevents $\boldsymbol{F}$ from dropping to a trivial solution $\boldsymbol{F} = \mathbf{0}_{n\times n}$. Further, we relax the constrained optimization problem in Eq. (4) to a unconstrained problem by absorbing the two original constraints to the objective function. Moreover, following the min-max approach [25], problem (4) can be reformulated as

$$\max_{\boldsymbol{\alpha}\in\mathcal{A}}\ \min_{\boldsymbol{F}\in\mathcal{S}_+^n}\ \mathbf{1}^\top\boldsymbol{\alpha} - \frac{1}{2}\boldsymbol{\alpha}^\top\boldsymbol{Y}\big(\boldsymbol{F}\odot\boldsymbol{K}\big)\boldsymbol{Y}\boldsymbol{\alpha} + \eta\|\boldsymbol{F}-\mathbf{1}\mathbf{1}^\top\|_{\text{F}}^2 + \tau\eta\|\boldsymbol{F}\|_*\,, \tag{5}$$

where η and τ are two regularization parameters. The nuclear norm $\|\cdot\|_*$ is the convex approximation of the rank function. Imposing these two regularizers and the positive semi-definiteness on $\boldsymbol{F}$, the solution to problem (5) can be restricted in a bounded set demonstrated by Lemma 2 (see the next page), so that we can seek for a good trade-off between the model flexibility and complexity. Specifically, in problem (5), the inner minimization problem with respect to $\boldsymbol{F}$ is a convex conic program, and the outer maximization problem is a point-wise minimum of concave quadratic functions of $\boldsymbol{\alpha}$. As a consequence, problem (5) is also convex to satisfy strong duality as discussed above.

The superiority of our DANK model is intuitively illustrated in Fig. 1, which comprehensively demonstrates that learning $\boldsymbol{F}$ can provide adequate model flexibility for SVM. Here we show the effectiveness of the introduced constraints on $\boldsymbol{F}$ for a trade-off between the model flexibility and complexity. In Fig. 2, we present the range of the optimal solution $\boldsymbol{F}^*$ generated by our DANK model in Eq. (5) with $\sigma = 1$ (left panel) and $\sigma = 0.1$ (right panel), respectively. One can see that, in these two cases, the values in $\boldsymbol{F}^*$

1. It is admitted by Schur Product Theorem [30] which relates positive semi-definite matrices to the Hadamard product.

range from 0.9 to 1.1, and we obtain $\text{rank}(\boldsymbol{F}^*) = 7$ (left panel) and $\text{rank}(\boldsymbol{F}^*) = 9$ (right panel), respectively. Thereby, such small fluctuation on $\boldsymbol{F}^*$ and its low rank structure effectively control its complexity, and make it easy to extend to test data.

To sum up, we formulate the proposed DANK model embedded in SVM as a max-min optimization problem including learning the adaptive matrix $\boldsymbol{F}$ and standard SVM parameters in problem (5). In the next section, we develop a smooth optimization algorithm to directly solve the DANK model in SVM.

3 Algorithm for DANK model in SVM

This section firstly investigates the gradient-Lipschitz continuity of the objective function in problem (5) and then introduces the Nesterov's smooth optimization method to solve it.

3.1 Gradient-Lipschitz continuity of DANK in SVM

Here we denote the objective function in Eq. (5) as

$$H(\boldsymbol{\alpha},\boldsymbol{F})=\mathbf{1}^\top\boldsymbol{\alpha}-\frac{1}{2}\boldsymbol{\alpha}^\top\boldsymbol{Y}(\boldsymbol{F}\odot\boldsymbol{K})\boldsymbol{Y}\boldsymbol{\alpha}+\eta\|\boldsymbol{F}-\mathbf{1}\mathbf{1}^\top\|_{\mathrm{F}}^2+\tau\eta\|\boldsymbol{F}\|_*\,,$$

of which the optimal solution $(\boldsymbol{\alpha}^*,\boldsymbol{F}^*)$ is a saddle point of $H(\boldsymbol{\alpha},\boldsymbol{F})$ due to the property of the max-min problem (5). It is easy to check $H(\boldsymbol{\alpha},\boldsymbol{F}^*)\le H(\boldsymbol{\alpha}^*,\boldsymbol{F}^*)\le H(\boldsymbol{\alpha}^*,\boldsymbol{F})$ for any feasible $\boldsymbol{\alpha}$ and $\boldsymbol{F}$. Further, we define the following function

$$h(\boldsymbol{\alpha})\triangleq H(\boldsymbol{\alpha},\boldsymbol{F}^*)=\min_{\boldsymbol{F}\in\mathcal{S}_+^n}H(\boldsymbol{\alpha},\boldsymbol{F})\,, \tag{6}$$

which is concave since h is the minimum of a sequence of concave functions. Accordingly, the function $H(\boldsymbol{\alpha},\boldsymbol{F})$ is the saddle representation of $h(\boldsymbol{\alpha})$.

The following lemma presents the differentiable property of the optimal value function, which would help to investigate the gradient-Lipschitz continuity of $h(\boldsymbol{\alpha})$.

Lemma 1. *([31], [32]) Given a function $g(\boldsymbol{x},\boldsymbol{u})$ on a metric space $\mathcal{X}$ and a normed space $\mathcal{U}$. Suppose that for any $\boldsymbol{x}\in\mathcal{X}$, the function $g(\boldsymbol{x},\cdot)$ is differentiable, $g(\boldsymbol{x},\boldsymbol{u})$ and $\nabla_{\boldsymbol{u}}g(\boldsymbol{x},\boldsymbol{u})$ are continuous on $\mathcal{X}\times\mathcal{U}$, and $\mathcal{B}$ is a compact subset of $\mathcal{X}$. Then the optimal value function $g(\boldsymbol{u}):=\inf_{\boldsymbol{x}\in\mathcal{B}}g(\boldsymbol{x},\boldsymbol{u})$ is differentiable. When the minimizer $\boldsymbol{x}(\boldsymbol{u})$ of $g(\cdot,\boldsymbol{u})$ is unique, the gradient is given by $\nabla g(\boldsymbol{u})=\nabla_{\boldsymbol{u}}g(\boldsymbol{u},\boldsymbol{x}(\boldsymbol{u}))$.*

Before applying Lemma 1 to our proof, we need the following two useful lemmas.

Lemma 2. *Problem* (6) *is equivalent to the following formulation*

$$h(\boldsymbol{\alpha})=\min_{\boldsymbol{F}\in\mathcal{B}}H(\boldsymbol{\alpha},\boldsymbol{F})\,, \tag{7}$$

where $\mathcal{B}:=\Big\{\boldsymbol{F}\in\mathcal{S}_+^n:\lambda_{\max}(\boldsymbol{F})\le n-\frac{\tau}{2}+\frac{nC^2}{4\eta}\lambda_{\max}(\boldsymbol{K})\Big\}$.

Proof. Defining

$$\boldsymbol{\Gamma}(\boldsymbol{\alpha})=\frac{1}{4\eta}\,\mathrm{diag}(\boldsymbol{\alpha}^\top\boldsymbol{Y})\boldsymbol{K}\,\mathrm{diag}(\boldsymbol{\alpha}^\top\boldsymbol{Y})\,, \tag{8}$$

problem (6) can be reformulated as

$$\min_{\boldsymbol{F}\in\mathcal{S}_+^n}\|\boldsymbol{F}-\mathbf{1}\mathbf{1}^\top-\boldsymbol{\Gamma}(\boldsymbol{\alpha})\|_{\mathrm{F}}^2+\tau\|\boldsymbol{F}\|_*\,, \tag{9}$$

where the regularization parameter η is implicitly included in $\boldsymbol{\Gamma}(\boldsymbol{\alpha})$. The optimizer of problem (9) is $\boldsymbol{F}^*=\mathcal{J}_{\frac{\tau}{2}}(\mathbf{1}\mathbf{1}^\top+\boldsymbol{\Gamma}(\boldsymbol{\alpha}))$, where $\mathcal{J}_\tau(\boldsymbol{A})=\boldsymbol{U}_{\boldsymbol{A}}\mathcal{S}_\tau(\boldsymbol{\Sigma}_{\boldsymbol{A}})\boldsymbol{V}_{\boldsymbol{A}}^\top$ is the singular value soft-thresholding operator with the singular value decomposition $\boldsymbol{A}=\boldsymbol{U}\boldsymbol{\Sigma}\boldsymbol{V}^\top$ and the soft-thresholding operator is $\mathcal{S}_\tau(\boldsymbol{A}_{ij})=\mathrm{sign}(\boldsymbol{A}_{ij})\max(0,|\boldsymbol{A}_{ij}|-\tau)$. Accordingly, we have

$$\begin{aligned}\lambda_{\max}(\boldsymbol{F})&=\lambda_{\max}\Big(\mathcal{J}_{\frac{\tau}{2}}\big(\mathbf{1}\mathbf{1}^\top+\boldsymbol{\Gamma}(\boldsymbol{\alpha})\big)\Big)=\lambda_{\max}\Big(\mathbf{1}\mathbf{1}^\top+\boldsymbol{\Gamma}(\boldsymbol{\alpha})\Big)-\frac{\tau}{2}\\
&\le\lambda_{\max}(\mathbf{1}\mathbf{1}^\top)+\frac{1}{4\eta}\lambda_{\max}\Big(\,\mathrm{diag}(\boldsymbol{\alpha}^\top\boldsymbol{Y})\boldsymbol{K}\,\mathrm{diag}(\boldsymbol{\alpha}^\top\boldsymbol{Y})\Big)-\frac{\tau}{2}\\
&=n+\frac{1}{4\eta}\Big\|\,\mathrm{diag}(\boldsymbol{\alpha}^\top\boldsymbol{Y})\boldsymbol{K}^{\frac{1}{2}}\Big\|_2-\frac{\tau}{2}\\
&\le n+\frac{1}{4\eta}\|\,\mathrm{diag}(\boldsymbol{\alpha}^\top\boldsymbol{Y})\|_2\|\boldsymbol{K}^{\frac{1}{2}}\|_2-\frac{\tau}{2}\\
&\le n-\frac{\tau}{2}+\frac{nC^2}{4\eta}\lambda_{\max}(\boldsymbol{K})\,,\end{aligned} \tag{10}$$

where the first inequality uses the property of maximum eigenvalues, *i.e.*, $\lambda_{\max}(\boldsymbol{A}+\boldsymbol{B})\le\lambda_{\max}(\boldsymbol{A})+\lambda_{\max}(\boldsymbol{B})$ for any $\boldsymbol{A},\boldsymbol{B}\in\mathcal{S}^n$, and the last inequality admits by $\|\boldsymbol{\alpha}\|^2\le nC^2$. □

Lemma 3. *For any $\boldsymbol{\alpha}_1$, $\boldsymbol{\alpha}_2\in\mathcal{A}$, we have*

$$\big\|\boldsymbol{F}(\boldsymbol{\alpha}_1)-\boldsymbol{F}(\boldsymbol{\alpha}_2)\big\|_{\mathrm{F}}\le\frac{\|\boldsymbol{K}\|\big(\|\boldsymbol{\alpha}_1\|+\|\boldsymbol{\alpha}_2\|\big)}{4\eta}\big\|\boldsymbol{\alpha}_1-\boldsymbol{\alpha}_2\big\|_2\,,$$

where $\boldsymbol{F}(\boldsymbol{\alpha}_1)=\mathcal{J}_{\frac{\tau}{2}}\big(\mathbf{1}\mathbf{1}^\top+\boldsymbol{\Gamma}(\boldsymbol{\alpha}_1)\big)$ and $\boldsymbol{F}(\boldsymbol{\alpha}_2)=\mathcal{J}_{\frac{\tau}{2}}\big(\mathbf{1}\mathbf{1}^\top+\boldsymbol{\Gamma}(\boldsymbol{\alpha}_2)\big)$.

Proof. Since $\mathcal{J}_{\frac{\tau}{2}}$ is non-expansive [33], *i.e.*, for any $\boldsymbol{\Omega}_1$ and $\boldsymbol{\Omega}_2$

$$\|\mathcal{J}_{\frac{\tau}{2}}(\boldsymbol{\Omega}_1)-\mathcal{J}_{\frac{\tau}{2}}(\boldsymbol{\Omega}_2)\|_{\mathrm{F}}\le\|\boldsymbol{\Omega}_1-\boldsymbol{\Omega}_2\|_{\mathrm{F}}$$

where $\boldsymbol{\Omega}_1=\mathbf{1}\mathbf{1}^\top+\boldsymbol{\Gamma}(\boldsymbol{\alpha}_1)$ and $\boldsymbol{\Omega}_2=\mathbf{1}\mathbf{1}^\top+\boldsymbol{\Gamma}(\boldsymbol{\alpha}_2)$. Thereby, we have

$$\begin{aligned}&\big\|\boldsymbol{F}(\boldsymbol{\alpha}_1)-\boldsymbol{F}(\boldsymbol{\alpha}_2)\big\|_{\mathrm{F}}\le\big\|\boldsymbol{\Gamma}(\boldsymbol{\alpha}_1)-\boldsymbol{\Gamma}(\boldsymbol{\alpha}_2)\big\|_{\mathrm{F}}\\
&=\frac{1}{4\eta}\Big\|\mathrm{diag}\big(\boldsymbol{\alpha}_1^\top\boldsymbol{Y}\big)\boldsymbol{K}\mathrm{diag}\big(\boldsymbol{\alpha}_1^\top\boldsymbol{Y}\big)-\mathrm{diag}\big(\boldsymbol{\alpha}_2^\top\boldsymbol{Y}\big)\boldsymbol{K}\mathrm{diag}\big(\boldsymbol{\alpha}_2^\top\boldsymbol{Y}\big)\Big\|_{\mathrm{F}}\\
&=\frac{1}{4\eta}\Big\|\,\mathrm{diag}\,\big(\boldsymbol{\alpha}_1{}^\top\boldsymbol{Y}+\boldsymbol{\alpha}_2{}^\top\boldsymbol{Y}\big)\boldsymbol{K}\,\mathrm{diag}\,\big(\boldsymbol{\alpha}_1{}^\top\boldsymbol{Y}-\boldsymbol{\alpha}_2{}^\top\boldsymbol{Y}\big)\Big\|_{\mathrm{F}}\\
&\le\frac{1}{4\eta}\|\boldsymbol{K}\|_{\mathrm{F}}\Big\|\,\mathrm{diag}\big(\boldsymbol{\alpha}_1{}^\top\boldsymbol{Y}+\boldsymbol{\alpha}_2{}^\top\boldsymbol{Y}\big)\Big\|_{\mathrm{F}}\Big\|\,\mathrm{diag}\,\big(\boldsymbol{\alpha}_1{}^\top\boldsymbol{Y}-\boldsymbol{\alpha}_2{}^\top\boldsymbol{Y}\big)\Big\|_{\mathrm{F}}\\
&\le\frac{\|\boldsymbol{K}\|_{\mathrm{F}}}{4\eta}\big\|\boldsymbol{\alpha}_1+\boldsymbol{\alpha}_2\big\|_2\big\|\boldsymbol{\alpha}_1-\boldsymbol{\alpha}_2\big\|_2\,,\end{aligned}$$

which yields the desired result. □

Lemma 2 demonstrates that the optimal solution $\boldsymbol{F}^*$ in Eq. (9) belongs to a bounded region in $\mathcal{S}_+^n$. Formally, we present the following theorem.

Theorem 1. *The function $h(\boldsymbol{\alpha})$ is gradient-Lipschitz continuous with Lipschitz constant $L=n+\frac{3nC^2\|\boldsymbol{K}\|^2}{4\eta}$, i.e., for any $\boldsymbol{\alpha}_1$, $\boldsymbol{\alpha}_2\in\mathcal{A}$, the inequality $\|\nabla h(\boldsymbol{\alpha}_1)-\nabla h(\boldsymbol{\alpha}_2)\|_2\le L\|\boldsymbol{\alpha}_1-\boldsymbol{\alpha}_2\|_2$ holds.*

Proof. The gradient of $h(\boldsymbol{\alpha})$ in Eq. (6) is computed as

$$\nabla h(\boldsymbol{\alpha})=\mathbf{1}-\boldsymbol{Y}\big(\boldsymbol{F}(\boldsymbol{\alpha})\odot\boldsymbol{K}\big)\boldsymbol{Y}\boldsymbol{\alpha}\,. \tag{11}$$

It is obvious that $\boldsymbol{F}(\boldsymbol{\alpha})$ is unique over the compact set $\mathcal{B}$. Hence, according to Lemma 1, for any $\boldsymbol{\alpha}_1$, $\boldsymbol{\alpha}_2\in\mathcal{A}$, from the representation of $\nabla h(\boldsymbol{\alpha})$ in Eq. (11), the function $h(\boldsymbol{\alpha})$ is proven to be gradient-Lipschitz continuous. More specifically, the Lipschitz constant is given by Eq. (12) (see the next page), which concludes the proof.

□

The above theoretical analyses provide a justification for utilizing a smooth optimization method to directly solve the DANK model embedded in SVM.

$$
\begin{aligned}
\|\nabla h(\boldsymbol{\alpha}_1)-\nabla h(\boldsymbol{\alpha}_2)\|_2 &= \left\|\boldsymbol{Y}\big(\boldsymbol{F}_1\odot\boldsymbol{K}\big)\boldsymbol{Y}\boldsymbol{\alpha}_1-\boldsymbol{Y}\big(\boldsymbol{F}_2\odot\boldsymbol{K}\big)\boldsymbol{Y}\boldsymbol{\alpha}_2\right\|_2 = \left\|\boldsymbol{Y}\big(\boldsymbol{F}_1-\boldsymbol{F}_2\big)\odot\boldsymbol{K}\boldsymbol{Y}\boldsymbol{\alpha}_1-\boldsymbol{Y}\big(\boldsymbol{F}_2\odot\boldsymbol{K}\big)\boldsymbol{Y}\big(\boldsymbol{\alpha}_2-\boldsymbol{\alpha}_1\big)\right\|_2 \\
&\le \left\|\boldsymbol{Y}\big(\boldsymbol{F}_1-\boldsymbol{F}_2\big)\odot\boldsymbol{K}\boldsymbol{Y}\boldsymbol{\alpha}_1\right\|_2+\left\|\boldsymbol{Y}\big(\boldsymbol{F}_2\odot\boldsymbol{K}\big)\boldsymbol{Y}\big(\boldsymbol{\alpha}_2-\boldsymbol{\alpha}_1\big)\right\|_2 \\
&\le \left\|\big(\boldsymbol{F}_1-\boldsymbol{F}_2\big)\odot\boldsymbol{K}\right\|_{\mathrm{F}}\|\boldsymbol{\alpha}_1\|_2+\left\|\boldsymbol{F}_2\odot\boldsymbol{K}\right\|_{\mathrm{F}}\|\boldsymbol{\alpha}_1-\boldsymbol{\alpha}_2\|_2 \\
&\le \left\|\boldsymbol{F}_1-\boldsymbol{F}_2\right\|_{\mathrm{F}}\|\boldsymbol{K}\|_{\mathrm{F}}\|\boldsymbol{\alpha}_1\|_2+\|\boldsymbol{F}_2\|_{\mathrm{F}}\|\boldsymbol{K}\|_{\mathrm{F}}\|\boldsymbol{\alpha}_1-\boldsymbol{\alpha}_2\|_2 \quad \text{(Using } \|\boldsymbol{A}\odot\boldsymbol{B}\|_{\mathrm{F}}\le \mathrm{Tr}(\boldsymbol{A}\boldsymbol{B}^\top)\le\|\boldsymbol{A}\|_{\mathrm{F}}\|\boldsymbol{B}\|_{\mathrm{F}}) \\
&\le \frac{\|\boldsymbol{K}\|_{\mathrm{F}}^2}{4\eta}\|\boldsymbol{\alpha}_1\|_2\big(\|\boldsymbol{\alpha}_1\|_2+\|\boldsymbol{\alpha}_2\|_2\big)\|\boldsymbol{\alpha}_1-\boldsymbol{\alpha}_2\|_2+\lambda_{\max}(\boldsymbol{F}_2)\|\boldsymbol{K}\|_{\mathrm{F}}\|\boldsymbol{\alpha}_1-\boldsymbol{\alpha}_2\|_2 \quad \text{(Using Lemma 3 and } \boldsymbol{F}_2\in\mathcal{B}) \\
&\le \frac{\|\boldsymbol{K}\|_{\mathrm{F}}^2}{4\eta}\|\boldsymbol{\alpha}_1\|_2\big(\|\boldsymbol{\alpha}_1\|_2+\|\boldsymbol{\alpha}_2\|_2\big)\|\boldsymbol{\alpha}_1-\boldsymbol{\alpha}_2\|_2+\Big(n+\frac{nC^2}{4\eta}\lambda_{\max}(\boldsymbol{K})\Big)\|\boldsymbol{K}\|_{\mathrm{F}}\|\boldsymbol{\alpha}_1-\boldsymbol{\alpha}_2\|_2 \quad \text{(Using Lemma 2)} \\
&\le \Big(n+\frac{3nC^2\|\boldsymbol{K}\|_{\mathrm{F}}^2}{4\eta}\Big)\|\boldsymbol{\alpha}_1-\boldsymbol{\alpha}_2\|_2 \quad \text{(Using } 0\le\boldsymbol{\alpha}\le C,\ \|\boldsymbol{\alpha}\|_2^2\le nC^2, \lambda_{\max}(\boldsymbol{K})\le\|\boldsymbol{K}\|_{\mathrm{F}})\,.
\end{aligned}
\tag{12}
$$

Algorithm 1: Projected gradient method with Nesterov's acceleration to solve problem (5)

Input: The kernel matrix $\boldsymbol{K}$, the label matrix $\boldsymbol{Y}$, and the Lipschitz constant $L=n+\frac{3nC^2\|\boldsymbol{K}\|^2}{4\eta}$

Output: The optimal $\boldsymbol{\alpha}^*$

1 Set the stopping criteria $t_{\max}=2000$ and $\epsilon=10^{-4}$.
2 Initialize $t=0$ and $\boldsymbol{\alpha}_0\in\mathcal{A}$.
3 **Repeat**
4 Compute $\boldsymbol{F}(\boldsymbol{\alpha}^{(t)})=\mathcal{J}_{\frac{\tau}{2}}\big(\boldsymbol{1}\boldsymbol{1}^\top+\boldsymbol{\Gamma}(\boldsymbol{\alpha}^{(t)})\big)$;
5 Compute $\nabla h(\boldsymbol{\alpha}^{(t)})=\boldsymbol{1}-\boldsymbol{Y}\big(\boldsymbol{F}(\boldsymbol{\alpha}^{(t)})\odot\boldsymbol{K}\big)\boldsymbol{Y}\boldsymbol{\alpha}^{(t)}$;
6 Compute $\boldsymbol{\theta}^{(t)}=\mathcal{P}_{\mathcal{A}}\Big(\boldsymbol{\alpha}^{(t)}+\frac{1}{L}\nabla h(\boldsymbol{\alpha}^{(t)})\Big)$;
7 Compute $\boldsymbol{\beta}^{(t)}=\mathcal{P}_{\mathcal{A}}\Big(\boldsymbol{\alpha}_0+\frac{1}{2L}\sum_{i=0}^{t}(i+1)\nabla h(\boldsymbol{\alpha}^{(t)})\Big)$;
8 Set $\boldsymbol{\alpha}^{(t+1)}=\frac{t+1}{t+3}\boldsymbol{\theta}^{(t)}+\frac{2}{t+3}\boldsymbol{\beta}^{(t)}$;
9 $t:=t+1$;
10 **Until** $t\ge t_{\max}$ *or* $\|\boldsymbol{\alpha}^{(t)}-\boldsymbol{\alpha}^{(t-1)}\|_2\le\epsilon$;

3.2 Nesterov's Smooth Optimization Method

In this section, we introduce a projected gradient algorithm with Nesterov's acceleration to solve the optimization problem (5). Nesterov [34] proposes an optimal scheme for smooth optimization $\min_{\boldsymbol{x}\in\mathcal{Q}} g(\boldsymbol{x})$, where g is a convex gradient-Lipschitz continuous function over a closed convex set $\mathcal{Q}$. Introducing a continuous and strongly convex function denoted as *prox-function* $d(\boldsymbol{x})$ on $\mathcal{Q}$, the first-order projected gradient method with Nesterov's acceleration can then be used to solve this problem.

In our model, we aim to solve the following convex problem

$$
\max_{\boldsymbol{\alpha}\in\mathcal{A}}\ \ h(\boldsymbol{\alpha})\,, \tag{13}
$$

where $h(\boldsymbol{\alpha})$ is concave gradient-Lipschitz continuous with the Lipschitz constant $L=n+\frac{3nC^2\|\boldsymbol{K}\|^2}{4\eta}$ as demonstrated in Theorem 1. Here the proxy-function is defined as $d(\boldsymbol{\alpha})=\frac{1}{2}\|\boldsymbol{\alpha}-\boldsymbol{\alpha}_0\|^2$ with $\boldsymbol{\alpha}_0\in\mathcal{A}$. The first-order Nesterov's smooth optimization method for solving problem (5) is summarized in Algorithm 1.

The key steps of Nesterov's acceleration are characterized by Lines 6, 7, and 8 in Algorithm 1. To be specific, according to [34], we need to solve the following problem

$$
\min_{\boldsymbol{\alpha}\in\mathcal{A}}\frac{L}{2}\|\boldsymbol{\alpha}-\boldsymbol{\alpha}_0\|^2+\sum_{i=1}^{t}\frac{i+1}{2}\Big[h(\boldsymbol{\alpha}_i)+\big\langle\nabla h(\boldsymbol{\alpha}^{(t)}),\boldsymbol{\alpha}-\boldsymbol{\alpha}_i\big\rangle\Big]\,,
$$

which is equivalent to

$$
\min_{\boldsymbol{\alpha}\in\mathcal{A}}\ \ \Big\|\boldsymbol{\alpha}-\boldsymbol{\alpha}_0-\frac{1}{2L}\sum_{i=0}^{t}(i+1)\nabla h(\boldsymbol{\alpha}^{(t)})\Big\|_2^2\,,
$$

of which the optimizer is $\mathcal{P}_{\mathcal{A}}\Big(\boldsymbol{\alpha}_0+\frac{1}{2L}\sum_{i=0}^{t}(i+1)\nabla h(\boldsymbol{\alpha}^{(t)})\Big)$ as outlined in Line 7 in Algorithm 1. Specifically, when Lines 6, 7, and 8 in Algorithm 1 are replaced by

$$
\boldsymbol{\alpha}^{(t+1)}=\mathcal{P}_{\mathcal{A}}\Big(\boldsymbol{\alpha}^{(t)}+\frac{1}{L}\nabla h(\boldsymbol{\alpha}^{(t)})\Big) \tag{14}
$$

with the Lipschitz constant $L=n-\frac{\tau}{2}+\frac{nC^2}{4\eta}\lambda_{\max}(\boldsymbol{K})$ derived from Lemma 2, the Nesterov's smooth method degenerates to a standard projected gradient method.

The convergence of the Nesterov smoothing optimization algorithm is pointed out by Theorem 2 in [34], namely

$$
h(\boldsymbol{\alpha}^*)-h(\boldsymbol{\beta}^{(t)})\le\frac{8L\|\boldsymbol{\alpha}_0-\boldsymbol{\alpha}^*\|^2}{(t+1)(t+2)}\,,
$$

where $\boldsymbol{\alpha}^*$ is the optimal solution of Eq. (13). Note that, in general, Algorithm 1 cannot guarantee $\{h(\boldsymbol{\alpha}^{(t)}):t\in\mathbb{N}\}$ and $\{h(\boldsymbol{\beta}^{(t)}):t\in\mathbb{N}\}$ to be monotone increasing during the maximization process. Nevertheless, such algorithm can be modified to obtain a monotone sequence with replacing Line 6 in Algorithm 1 by

$$
\begin{cases}
\tilde{\boldsymbol{\theta}}^{(t)}=\mathcal{P}_{\mathcal{A}}\Big(\boldsymbol{\alpha}^{(t)}+\dfrac{1}{L}\nabla h(\boldsymbol{\alpha}^{(t)})\Big)\,,\\[2ex]
\boldsymbol{\theta}^{(t)}=\operatorname*{argmax}_{\boldsymbol{\alpha}}h(\boldsymbol{\alpha}),\ \boldsymbol{x}\in\{\boldsymbol{\theta}^{(t-1)},\tilde{\boldsymbol{\theta}}^{(t)},\boldsymbol{\alpha}^{(t)}\}\,.
\end{cases}
$$

3.3 Discussion

Here we briefly discuss the computational complexity of the developed algorithm and out-of-sample extension for the test data.

The applied projected gradient algorithm with Nesterov's acceleration in Algorithm 1 contains the following steps. An eigenvalue decomposition in Line 4 is conducted with $\mathcal{O}(n^3)$ complexity, which can be further improved to $\mathcal{O}(n^2)$ by an Arnoldi process [35]. In Lines 6 and 7, the projection on a convex set $\mathcal{A}$ costs $\mathcal{O}(n\log n)$ to compute $\boldsymbol{\theta}^{(k)}$ and $\boldsymbol{\beta}^{(k)}$. The Nesterov's smooth optimization method takes $\mathcal{O}(\sqrt{L/\epsilon})$ to find an ϵ-optimal solution, which is better than the standard projected gradient method with the complexity $\mathcal{O}(L/\epsilon)$. Finally, the overall complexity for solving the DANK model in SVM is $\mathcal{O}(n^2\sqrt{L/\epsilon})$.

Besides, here we present how to address out-of-sample extensions in our DANK model. This problem is a common issue in

nonparametric kernel learning [27], [28], [29]. In our model, the flexible kernel matrix $\boldsymbol{F} \odot \boldsymbol{K}$ is learned from the training data in a nonparametric manner, while the adaptive kernel matrix for test data is unknown. To be specific, given the optimal $\boldsymbol{F}^*$ on training data, the test data $\mathcal{X}' = \{\boldsymbol{x}_i'\}_{i=1}^m$, and the initial kernel matrix for test data $\boldsymbol{K}' = [k(\boldsymbol{x}_i, \boldsymbol{x}_j')]_{n\times m}$, we aim to establish the adaptive matrix $\boldsymbol{F}'$ for test data. To this end, we use a simple but effective technique, *i.e.*, the nearest neighbor scheme to acquire $\boldsymbol{F}'$. Formally, $\boldsymbol{F}'$ is set by

$$\boldsymbol{F}_i' \leftarrow \boldsymbol{F}_{j^*}^*, \text{ if } \boldsymbol{x}_{j^*} = \mathcal{N}(\boldsymbol{x}_i', \mathcal{X})\,.$$

That is to say, if $\boldsymbol{x}_{j^*}$ is the nearest neighbor of $\boldsymbol{x}_i'$ among the training data set $\mathcal{X}$, the j^*-th column of $\boldsymbol{F}^*$ is assigned to the i-th column of $\boldsymbol{F}'$. By doing so, $\boldsymbol{F}^*$ is directly extended to $\boldsymbol{F}'$, resulting in the flexible kernel matrix $\boldsymbol{F}' \odot \boldsymbol{K}'$ for test data. Admittedly, there might leave over the inconsistency between the training kernel and the test kernel. Nevertheless, as mentioned in Eq. (5), $\boldsymbol{F}$ is designed to vary in a small range, and is expected to smoothly vary between any two neighboring data points in $\mathcal{F}$, so the extension to $\boldsymbol{F}'$ on test data by such scheme is reasonable.

4 DANK Model in SVR

In this section, we incorporate the DANK model into SVR for regression and also develop the Nesterov's smooth optimization algorithm to solve it.

Similar to DANK in SVM revealed by problem (5), we incorporate the DANK model into SVR with the ε-insensitive loss, namely

$$\begin{aligned}\max_{\hat{\boldsymbol{\alpha}},\check{\boldsymbol{\alpha}}} \min_{\boldsymbol{F}\in\mathcal{S}_+^n} & -\frac{1}{2}(\hat{\boldsymbol{\alpha}}-\check{\boldsymbol{\alpha}})^\top(\boldsymbol{F}\odot\boldsymbol{K})(\hat{\boldsymbol{\alpha}}-\check{\boldsymbol{\alpha}}) + (\hat{\boldsymbol{\alpha}}-\check{\boldsymbol{\alpha}})^\top\boldsymbol{y} \\ & - \varepsilon(\hat{\boldsymbol{\alpha}}+\check{\boldsymbol{\alpha}})^\top\mathbf{1} + \eta\|\boldsymbol{F}-\mathbf{1}\mathbf{1}^\top\|_{\mathrm{F}}^2 + \tau\eta\|\boldsymbol{F}\|_* \\ \text{s.t. } & 0 \le \hat{\boldsymbol{\alpha}}, \check{\boldsymbol{\alpha}} \le C,\ (\hat{\boldsymbol{\alpha}}-\check{\boldsymbol{\alpha}})^\top\boldsymbol{y} = 0\,,\end{aligned} \tag{15}$$

where the dual variable is $\boldsymbol{\alpha} = \hat{\boldsymbol{\alpha}} - \check{\boldsymbol{\alpha}}$. The objective function in problem (15) is denoted as $H(\hat{\boldsymbol{\alpha}}, \check{\boldsymbol{\alpha}}, \boldsymbol{F})$. Further, we define the following function

$$h(\hat{\boldsymbol{\alpha}}, \check{\boldsymbol{\alpha}}) \triangleq H(\hat{\boldsymbol{\alpha}}, \check{\boldsymbol{\alpha}}, \boldsymbol{F}^*) = \min_{\boldsymbol{F}\in\mathcal{S}_+^n} H(\hat{\boldsymbol{\alpha}}, \check{\boldsymbol{\alpha}}, \boldsymbol{F})\,, \tag{16}$$

where $h(\hat{\boldsymbol{\alpha}}, \check{\boldsymbol{\alpha}})$ can be obtained by solving the following problem

$$\min_{\boldsymbol{F}\in\mathcal{S}_+^n} \|\boldsymbol{F} - \boldsymbol{\Gamma}(\hat{\boldsymbol{\alpha}}, \check{\boldsymbol{\alpha}})\|_{\mathrm{F}}^2 + \tau\|\boldsymbol{F}\|_*\,, \tag{17}$$

with $\boldsymbol{\Gamma}(\hat{\boldsymbol{\alpha}}, \check{\boldsymbol{\alpha}}) = \frac{1}{4\eta}\operatorname{diag}(\hat{\boldsymbol{\alpha}}-\check{\boldsymbol{\alpha}})^\top\boldsymbol{K}\operatorname{diag}(\hat{\boldsymbol{\alpha}}-\check{\boldsymbol{\alpha}})$. The optimal solution of Eq. (17) is $\boldsymbol{F}^* = \mathcal{J}_{\frac{\tau}{2}}(\mathbf{1}\mathbf{1}^\top + \boldsymbol{\Gamma}(\hat{\boldsymbol{\alpha}}, \check{\boldsymbol{\alpha}}))$. We can easily check that Lemma 2 is also applicable to problem (16)

$$h(\hat{\boldsymbol{\alpha}}, \check{\boldsymbol{\alpha}}) = \min_{\boldsymbol{F}\in\mathcal{B}} H(\hat{\boldsymbol{\alpha}}, \check{\boldsymbol{\alpha}}, \boldsymbol{F})\,.$$

Similar to Lemma 3, in our DANK model embedded in SVR, for any $\hat{\boldsymbol{\alpha}}_1, \check{\boldsymbol{\alpha}}_1, \hat{\boldsymbol{\alpha}}_2, \check{\boldsymbol{\alpha}}_2 \in \mathcal{A}$, $\|\boldsymbol{\Gamma}(\hat{\boldsymbol{\alpha}}_1, \check{\boldsymbol{\alpha}}_1) - \boldsymbol{\Gamma}(\hat{\boldsymbol{\alpha}}_2, \check{\boldsymbol{\alpha}}_2)\|_2$ can be bounded by the following lemma.

Lemma 4. *For any $\hat{\boldsymbol{\alpha}}_1, \check{\boldsymbol{\alpha}}_1, \hat{\boldsymbol{\alpha}}_2, \check{\boldsymbol{\alpha}}_2 \in \mathcal{A}$, we have*

$$\begin{aligned}&\left\|\boldsymbol{F}(\hat{\boldsymbol{\alpha}}_1, \check{\boldsymbol{\alpha}}_1) - \boldsymbol{F}(\hat{\boldsymbol{\alpha}}_2, \check{\boldsymbol{\alpha}}_2)\right\|_{\mathrm{F}} \le \|\boldsymbol{\Gamma}(\hat{\boldsymbol{\alpha}}_1, \check{\boldsymbol{\alpha}}_1) - \boldsymbol{\Gamma}(\hat{\boldsymbol{\alpha}}_2, \check{\boldsymbol{\alpha}}_2)\|_{\mathrm{F}} \\ &\le \frac{\|\boldsymbol{K}\|}{4\eta}\left\|\hat{\boldsymbol{\alpha}}_1 - \check{\boldsymbol{\alpha}}_1 + \hat{\boldsymbol{\alpha}}_2 - \check{\boldsymbol{\alpha}}_2\right\|_2\left\|\hat{\boldsymbol{\alpha}}_1 - \check{\boldsymbol{\alpha}}_1 - \hat{\boldsymbol{\alpha}}_2 + \check{\boldsymbol{\alpha}}_2\right\|_2\,,\end{aligned}$$

where $\boldsymbol{F}(\hat{\boldsymbol{\alpha}}_1, \check{\boldsymbol{\alpha}}_1)) = \mathcal{J}_{\frac{\tau}{2}}\big(\mathbf{1}\mathbf{1}^\top + \boldsymbol{\Gamma}(\hat{\boldsymbol{\alpha}}_1, \check{\boldsymbol{\alpha}}_1)\big)$ and $\boldsymbol{F}(\hat{\boldsymbol{\alpha}}_2, \check{\boldsymbol{\alpha}}_2) = \mathcal{J}_{\frac{\tau}{2}}\big(\mathbf{1}\mathbf{1}^\top + \boldsymbol{\Gamma}(\hat{\boldsymbol{\alpha}}_2, \check{\boldsymbol{\alpha}}_2)\big)$.

The proof of Lemma 4 is similar to that of Lemma 3, and here we omit the detailed proof. Next we present the partial derivative of $h(\hat{\boldsymbol{\alpha}}, \check{\boldsymbol{\alpha}})$ regarding to $\hat{\boldsymbol{\alpha}}$ and $\check{\boldsymbol{\alpha}}$.

Proposition 1. *The objective function $h(\hat{\boldsymbol{\alpha}}, \check{\boldsymbol{\alpha}})$ with two variables defined by Eq.* (16) *is differentiable and its partial derivatives are given by*

$$\begin{cases} \dfrac{\partial h(\hat{\boldsymbol{\alpha}}, \check{\boldsymbol{\alpha}})}{\partial \hat{\boldsymbol{\alpha}}} = -\varepsilon\boldsymbol{I} - (\hat{\boldsymbol{\alpha}} - \check{\boldsymbol{\alpha}})\boldsymbol{F}\odot\boldsymbol{K} + \boldsymbol{y}\,, \\ \dfrac{\partial h(\hat{\boldsymbol{\alpha}}, \check{\boldsymbol{\alpha}})}{\partial \check{\boldsymbol{\alpha}}} = -\varepsilon\boldsymbol{I} - (\hat{\boldsymbol{\alpha}} - \check{\boldsymbol{\alpha}})\boldsymbol{F}\odot\boldsymbol{K} - \boldsymbol{y}\,. \end{cases} \tag{18}$$

Formally, $h(\hat{\boldsymbol{\alpha}}, \check{\boldsymbol{\alpha}})$ is proven to be gradient-Lipschitz continuous by the following theorem.

Theorem 2. *The function $h(\hat{\boldsymbol{\alpha}}, \check{\boldsymbol{\alpha}})$ with its partial derivatives in Eq.* (18) *is gradient-Lipschitz continuous with the Lipschitz constant $L = 2\Big(n + \frac{9nC^2\|\boldsymbol{K}\|^2}{4\eta}\Big)$, i.e., for any $\hat{\boldsymbol{\alpha}}_1, \check{\boldsymbol{\alpha}}_1, \hat{\boldsymbol{\alpha}}_2, \check{\boldsymbol{\alpha}}_2 \in \mathcal{A}$, let the concentration vectors be $\tilde{\boldsymbol{\alpha}}_1 = [\hat{\boldsymbol{\alpha}}_1^\top, \check{\boldsymbol{\alpha}}_1^\top]^\top$ and $\tilde{\boldsymbol{\alpha}}_2 = [\hat{\boldsymbol{\alpha}}_2^\top, \check{\boldsymbol{\alpha}}_2^\top]^\top$, and the partial derivatives be*

$$\nabla_{\tilde{\boldsymbol{\alpha}}_1} h(\hat{\boldsymbol{\alpha}}_1, \check{\boldsymbol{\alpha}}_1) = \left[\Big(\frac{\partial h(\hat{\boldsymbol{\alpha}}, \check{\boldsymbol{\alpha}}_1)}{\partial \hat{\boldsymbol{\alpha}}}\Big|_{\hat{\boldsymbol{\alpha}}=\hat{\boldsymbol{\alpha}}_1}\Big)^\top, \Big(\frac{\partial h(\hat{\boldsymbol{\alpha}}_1, \check{\boldsymbol{\alpha}})}{\partial \check{\boldsymbol{\alpha}}}\Big|_{\check{\boldsymbol{\alpha}}=\check{\boldsymbol{\alpha}}_1}\Big)^\top\right]^\top,$$

$$\nabla_{\tilde{\boldsymbol{\alpha}}_2} h(\hat{\boldsymbol{\alpha}}_2, \check{\boldsymbol{\alpha}}_2) = \left[\Big(\frac{\partial h(\hat{\boldsymbol{\alpha}}, \check{\boldsymbol{\alpha}}_2)}{\partial \hat{\boldsymbol{\alpha}}}\Big|_{\hat{\boldsymbol{\alpha}}=\hat{\boldsymbol{\alpha}}_2}\Big)^\top, \Big(\frac{\partial h(\hat{\boldsymbol{\alpha}}_2, \check{\boldsymbol{\alpha}})}{\partial \check{\boldsymbol{\alpha}}}\Big|_{\check{\boldsymbol{\alpha}}=\check{\boldsymbol{\alpha}}_2}\Big)^\top\right]^\top,$$

we have

$$\|\nabla_{\tilde{\boldsymbol{\alpha}}_1} h(\hat{\boldsymbol{\alpha}}_1, \check{\boldsymbol{\alpha}}_1) - \nabla_{\tilde{\boldsymbol{\alpha}}_2} h(\hat{\boldsymbol{\alpha}}_2, \check{\boldsymbol{\alpha}}_2)\|_2 \le 2L\Big(\|\hat{\boldsymbol{\alpha}}_2 - \hat{\boldsymbol{\alpha}}_1\|_2 + \|\check{\boldsymbol{\alpha}}_2 - \check{\boldsymbol{\alpha}}_1\|_2\Big)\,.$$

Proof. For any $\hat{\boldsymbol{\alpha}}_1, \check{\boldsymbol{\alpha}}_1, \hat{\boldsymbol{\alpha}}_2, \check{\boldsymbol{\alpha}}_2 \in \mathcal{A}$, from the representation of $\nabla_{\hat{\boldsymbol{\alpha}}} h(\hat{\boldsymbol{\alpha}}, \check{\boldsymbol{\alpha}})$ in Proposition 1, we have

$$\left\|\frac{\partial h(\hat{\boldsymbol{\alpha}}, \check{\boldsymbol{\alpha}}_1)}{\partial \hat{\boldsymbol{\alpha}}}\Big|_{\hat{\boldsymbol{\alpha}}=\hat{\boldsymbol{\alpha}}_1} - \frac{\partial h(\hat{\boldsymbol{\alpha}}, \check{\boldsymbol{\alpha}}_2)}{\partial \hat{\boldsymbol{\alpha}}}\Big|_{\hat{\boldsymbol{\alpha}}=\hat{\boldsymbol{\alpha}}_2}\right\| \le L\Big(\|\hat{\boldsymbol{\alpha}}_2 - \hat{\boldsymbol{\alpha}}_1\|_2 + \|\check{\boldsymbol{\alpha}} - \check{\boldsymbol{\alpha}}_1\|\Big),$$

which is derived by Eq. (19) (see the next page). Similarly, $\left\|\frac{\partial h(\hat{\boldsymbol{\alpha}}_1, \check{\boldsymbol{\alpha}})}{\partial \check{\boldsymbol{\alpha}}}\Big|_{\check{\boldsymbol{\alpha}}=\check{\boldsymbol{\alpha}}_1} - \frac{\partial h(\hat{\boldsymbol{\alpha}}_2, \check{\boldsymbol{\alpha}})}{\partial \check{\boldsymbol{\alpha}}}\Big|_{\check{\boldsymbol{\alpha}}=\check{\boldsymbol{\alpha}}_2}\right\|_2$ can also be bounded. Combining these two inequalities, we complete the proof demonstrated by Eq. (20) (see the next page).

□

Based on the gradient-Lipschitz continuity of $h(\hat{\boldsymbol{\alpha}}, \check{\boldsymbol{\alpha}})$ demonstrated by Theorem 2, we are ready to present the first-order Nesterov's smooth optimization method for problem (15). The smooth optimization algorithm is summarized in Algorithm 2.

5 DANK in Large Scale Case

Scalability in kernel methods is a vital issue which often limits their applications in large datasets [36], [37], [38]. For example, the representative nonparametric kernel learning framework [18] is conducted by semi-definite programming with rough $\mathcal{O}(n^{6.5})$ complexity [20], which makes itself infeasible to large-scale datasets. Hence, in this section, we investigate kernel approximation in nonparametric kernel learning, and take our DANK model embedded in SVM as an example to illustrate this process. The presented theoretical results in this section are also suitable for the DANK model in SVR and other nonparametric kernel learning based algorithms.

To consider the scalability of our DANK model embedded in SVM, we reformulate problem (5) as

$$\begin{aligned}\max_{\boldsymbol{\alpha}} \min_{\boldsymbol{F}\in\mathcal{S}_+^n} & \mathbf{1}^\top\boldsymbol{\alpha} - \frac{1}{2}\boldsymbol{\alpha}^\top\boldsymbol{Y}(\boldsymbol{F}\odot\boldsymbol{K})\boldsymbol{Y}\boldsymbol{\alpha} + \eta\|\boldsymbol{F} - \mathbf{1}\mathbf{1}^\top\|_{\mathrm{F}}^2 \\ \text{s.t. } & \mathbf{0} \le \boldsymbol{\alpha} \le C\mathbf{1}\,.\end{aligned} \tag{21}$$

$$
\begin{aligned}
\Big\| \frac{\partial h(\hat{\boldsymbol{\alpha}}, \check{\boldsymbol{\alpha}}_1)}{\partial \hat{\boldsymbol{\alpha}}}\Big|_{\hat{\boldsymbol{\alpha}}=\hat{\boldsymbol{\alpha}}_1} - \frac{\partial h(\hat{\boldsymbol{\alpha}}, \check{\boldsymbol{\alpha}}_2)}{\partial \hat{\boldsymbol{\alpha}}}\Big|_{\hat{\boldsymbol{\alpha}}=\hat{\boldsymbol{\alpha}}_2} \Big\|_2 &= \big\| (\boldsymbol{F}_1 - \boldsymbol{F}_2) \odot \boldsymbol{K}(\hat{\boldsymbol{\alpha}}_1 - \check{\boldsymbol{\alpha}}_1) - (\boldsymbol{F}_2 \odot \boldsymbol{K})(\hat{\boldsymbol{\alpha}}_2 - \check{\boldsymbol{\alpha}}_2 - \hat{\boldsymbol{\alpha}}_1 + \check{\boldsymbol{\alpha}}_1) \big\|_2 \\
&\leq \big\| \boldsymbol{F}_1 - \boldsymbol{F}_2 \big\|_{\mathrm{F}} \|\boldsymbol{K}\|_{\mathrm{F}} \|\hat{\boldsymbol{\alpha}}_1 - \check{\boldsymbol{\alpha}}_1\|_2 + \|\boldsymbol{F}_2\|_{\mathrm{F}} \|\boldsymbol{K}\|_{\mathrm{F}} \|\hat{\boldsymbol{\alpha}}_2 - \check{\boldsymbol{\alpha}}_2 - \hat{\boldsymbol{\alpha}}_1 + \check{\boldsymbol{\alpha}}_1\|_2 \\
&\leq \Big(\frac{\|\boldsymbol{K}\|_{\mathrm{F}}^2}{4\eta} \|\hat{\boldsymbol{\alpha}}_1 - \check{\boldsymbol{\alpha}}_1\|_2 \|\hat{\boldsymbol{\alpha}}_2 - \check{\boldsymbol{\alpha}}_2 + \hat{\boldsymbol{\alpha}}_1 - \check{\boldsymbol{\alpha}}_1\|_2 + \lambda_{\max}(\boldsymbol{F}_2) \|\boldsymbol{K}\|_{\mathrm{F}} \Big) \|\hat{\boldsymbol{\alpha}}_2 - \check{\boldsymbol{\alpha}}_2 - \hat{\boldsymbol{\alpha}}_1 + \check{\boldsymbol{\alpha}}_1\|_2 \\
&\leq \Big(\frac{8nC^2 \|\boldsymbol{K}\|_{\mathrm{F}}^2}{4\eta} + n + \frac{nC^2}{4\eta} \lambda_{\max}(\boldsymbol{K}) \|\boldsymbol{K}\|_{\mathrm{F}} \Big) \Big(\|\hat{\boldsymbol{\alpha}}_2 - \hat{\boldsymbol{\alpha}}_1\|_2 + \|\check{\boldsymbol{\alpha}}_2 - \check{\boldsymbol{\alpha}}_1\|_2 \Big) \\
&\leq \Big(n + \frac{9nC^2 \|\boldsymbol{K}\|_{\mathrm{F}}^2}{4\eta} \Big) \Big(\|\hat{\boldsymbol{\alpha}}_2 - \hat{\boldsymbol{\alpha}}_1\|_2 + \|\check{\boldsymbol{\alpha}}_2 - \check{\boldsymbol{\alpha}}_1\|_2 \Big) .
\end{aligned} \tag{19}
$$

$$
\begin{aligned}
\|\nabla_{\tilde{\boldsymbol{\alpha}}_1} h(\hat{\boldsymbol{\alpha}}_1, \check{\boldsymbol{\alpha}}_1) - \nabla_{\tilde{\boldsymbol{\alpha}}_2} h(\hat{\boldsymbol{\alpha}}_2, \check{\boldsymbol{\alpha}}_2)\|_2 &\leq \Big\| \frac{\partial h(\hat{\boldsymbol{\alpha}}, \check{\boldsymbol{\alpha}}_1)}{\partial \hat{\boldsymbol{\alpha}}}\Big|_{\hat{\boldsymbol{\alpha}}=\hat{\boldsymbol{\alpha}}_1} - \frac{\partial h(\hat{\boldsymbol{\alpha}}, \check{\boldsymbol{\alpha}}_2)}{\partial \hat{\boldsymbol{\alpha}}}\Big|_{\hat{\boldsymbol{\alpha}}=\hat{\boldsymbol{\alpha}}_2} \Big\|_2 + \Big\| \frac{\partial h(\hat{\boldsymbol{\alpha}}_1, \check{\boldsymbol{\alpha}})}{\partial \check{\boldsymbol{\alpha}}}\Big|_{\check{\boldsymbol{\alpha}}=\check{\boldsymbol{\alpha}}_1} - \frac{\partial h(\hat{\boldsymbol{\alpha}}_2, \check{\boldsymbol{\alpha}})}{\partial \check{\boldsymbol{\alpha}}}\Big|_{\check{\boldsymbol{\alpha}}=\check{\boldsymbol{\alpha}}_2} \Big\|_2 \\
&\leq 2L \Big(\|\hat{\boldsymbol{\alpha}}_2 - \hat{\boldsymbol{\alpha}}_1\|_2 + \|\check{\boldsymbol{\alpha}}_2 - \check{\boldsymbol{\alpha}}_1\|_2 \Big) .
\end{aligned} \tag{20}
$$

Algorithm 2: Projected gradient method with Nesterov's acceleration to solve problem (15)

Input: The kernel matrix $\boldsymbol{K}$, the label matrix $\boldsymbol{Y}$, and the Lipschitz constant $L = 2\Big(n + \frac{9nC^2\|\boldsymbol{K}\|^2}{4\eta}\Big)$

Output: The optimal $\boldsymbol{\alpha}^*$

Set the stopping criteria $t_{\max} = 2000$ and $\epsilon = 10^{-4}$.
Initialize $t = 0$ and $\hat{\boldsymbol{\alpha}}_0, \check{\boldsymbol{\alpha}}_0 \in \mathcal{A}$.
Repeat
  Compute $\boldsymbol{F}(\hat{\boldsymbol{\alpha}}^{(t)}, \check{\boldsymbol{\alpha}}^{(t)}) = \mathcal{J}_{\frac{\tau}{2}}\big(\mathbf{1}\mathbf{1}^\top + \boldsymbol{\Gamma}(\hat{\boldsymbol{\alpha}}^{(t)}, \check{\boldsymbol{\alpha}}^{(t)})\big)$;
  Compute $\partial h / \partial \hat{\boldsymbol{\alpha}}$ and $\partial h / \partial \check{\boldsymbol{\alpha}}$ by Eq. (18), and concentrate them as $\nabla h(\hat{\boldsymbol{\alpha}}^{(t)}, \check{\boldsymbol{\alpha}}^{(t)}) = [(\partial h/\partial \hat{\boldsymbol{\alpha}})^\top, (\partial h/\partial \check{\boldsymbol{\alpha}})^\top]^\top$;
  Compute $\boldsymbol{\theta}^{(t)} = \mathcal{P}_{\mathcal{A}}\Big([\hat{\boldsymbol{\alpha}}^{(t)\top}, \check{\boldsymbol{\alpha}}^{(t)\top}]^\top + \frac{1}{2L}\nabla h(\hat{\boldsymbol{\alpha}}^{(t)}, \check{\boldsymbol{\alpha}}^{(t)})\Big)$;
  Compute $\boldsymbol{\beta}^{(t)} = \mathcal{P}_{\mathcal{A}}\Big([\hat{\boldsymbol{\alpha}}_0^\top, \check{\boldsymbol{\alpha}}_0^\top]^\top + \frac{1}{4L}\sum_{i=0}^{t}(i+1)\nabla h(\hat{\boldsymbol{\alpha}}^{(k)}, \check{\boldsymbol{\alpha}}^{(t)})\Big)$;
  Set $[\hat{\boldsymbol{\alpha}}^{(t+1)\top}, \check{\boldsymbol{\alpha}}^{(t+1)\top}]^\top = \frac{t+1}{t+3}\boldsymbol{\theta}^{(t)} + \frac{2}{t+3}\boldsymbol{\beta}^{(t)}$;
  Set $\boldsymbol{\alpha}^{(t+1)} = \hat{\boldsymbol{\alpha}}^{(t+1)} - \check{\boldsymbol{\alpha}}^{(t+1)}$ and $t := t + 1$;
Until $t \geq t_{\max}$ *or* $\|\boldsymbol{\alpha}^{(t)} - \boldsymbol{\alpha}^{(t-1)}\|_2 \leq \epsilon$;

Here we omit the low rank regularizer on $\boldsymbol{F}$ due to its inseparable property, which is based on the rapid decaying spectra of the kernel matrix [39]. In Section 6.1.5, we will verify that this term can be directly dropped without sacrificing too much performance in large-scale situations.

In our decomposition-based scalable approach, we divide the data into small subsets by k-means, and then solve each subset independently and efficiently. Such similar idea also exists in [40], [41], [42]. To be specific, we firstly partition the data into v subsets $\{\mathcal{V}_1, \mathcal{V}_2, \ldots, \mathcal{V}_v\}$, and then solve the respective sub-problems independently with the following formulation

$$
\begin{aligned}
\max_{\boldsymbol{\alpha}^{(c)}} \min_{\boldsymbol{F}^{(c,c)} \in \mathcal{S}_+^{|\mathcal{V}_c|}} & \mathbf{1}^\top \boldsymbol{\alpha}^{(c)} + \eta \|\boldsymbol{F}^{(c,c)} - \mathbf{1}\mathbf{1}^\top\|_{\mathrm{F}}^2 \\
& - \frac{1}{2} \boldsymbol{\alpha}^{(c)\top} \boldsymbol{Y}^{(c,c)} \big(\boldsymbol{F}^{(c,c)} \odot \boldsymbol{K}^{(c,c)}\big) \boldsymbol{Y}^{(c,c)} \boldsymbol{\alpha}^{(c)} \\
\text{s.t.} \ & \mathbf{0} \leq \boldsymbol{\alpha}^{(c)} \leq C\mathbf{1}, \ \ \forall\, c = 1, 2, \ldots, v ,
\end{aligned} \tag{22}
$$

where $|\mathcal{V}_c|$ denotes the number of data points in $\mathcal{V}_c$. Suppose that $(\bar{\boldsymbol{\alpha}}^{(c)}, \bar{\boldsymbol{F}}^{(c,c)})$ is the optimal solution of the c-th subproblem, the approximation solution $(\bar{\boldsymbol{\alpha}}, \bar{\boldsymbol{F}})$ to the whole problem is concatenated by $\bar{\boldsymbol{\alpha}} = [\bar{\boldsymbol{\alpha}}^{(1)}, \bar{\boldsymbol{\alpha}}^{(2)}, \ldots, \bar{\boldsymbol{\alpha}}^{(v)}]$ and $\bar{\boldsymbol{F}} = \mathrm{diag}(\bar{\boldsymbol{F}}^{(1,1)}, \bar{\boldsymbol{F}}^{(2,2)}, \ldots, \bar{\boldsymbol{F}}^{(v,v)})$, where $\bar{\boldsymbol{F}}$ is a block-diagonal matrix.

The above kernel approximation scheme makes the nonparametric kernel learning framework feasible to large-scale situations. In the next, we theoretically demonstrate the decomposition-based scalable approach in three aspects. First, the objective function value $H(\bar{\boldsymbol{\alpha}}, \bar{\boldsymbol{F}})$ in Eq. (21) is close to $H(\boldsymbol{\alpha}^*, \boldsymbol{F}^*)$. Second, the approximation solution $(\bar{\boldsymbol{\alpha}}, \bar{\boldsymbol{F}})$ is close to the optimal solution $(\boldsymbol{\alpha}^*, \boldsymbol{F}^*)$. Third, if $\boldsymbol{x}_i$ is not a support vector of the subproblem, it will also be a non-support vector of the whole problem under some conditions. To prove the above three propositions, we need the following lemma that links the subproblems to the whole problem.

Lemma 5. $(\bar{\boldsymbol{\alpha}}, \bar{\boldsymbol{F}})$ *is the optimal solution of the following problem*

$$
\begin{aligned}
\max_{\boldsymbol{\alpha}} \min_{\boldsymbol{F} \in \mathcal{S}_+^n} & \mathbf{1}^\top \boldsymbol{\alpha} - \frac{1}{2} \boldsymbol{\alpha}^\top \boldsymbol{Y} \big(\boldsymbol{F} \odot \bar{\boldsymbol{K}}\big) \boldsymbol{Y} \boldsymbol{\alpha} + \eta \|\boldsymbol{F} - \mathbf{1}\mathbf{1}^\top\|_{\mathrm{F}}^2 \\
\text{s.t.} \ & \mathbf{0} \leq \boldsymbol{\alpha} \leq C\mathbf{1} ,
\end{aligned} \tag{23}
$$

with the kernel $\bar{\boldsymbol{K}}$ *defined by*

$$
\bar{\boldsymbol{K}}_{ij} = I(\pi(\boldsymbol{x}_i), \pi(\boldsymbol{x}_j)) \boldsymbol{K}_{ij} ,
$$

where $\pi(\boldsymbol{x}_i)$ *is the cluster that* $\boldsymbol{x}_i$ *belongs to, and* $I(a, b) = 1$ *iff* $a = b$*, and* $I(a, b) = 0$ *otherwise.*

Proof. The quadratic term with respect to $\boldsymbol{\alpha}$ in Eq. (21) can be decomposed into

$$
\boldsymbol{\alpha}^\top \boldsymbol{Y} \big(\boldsymbol{F} \odot \bar{\boldsymbol{K}}\big) \boldsymbol{Y} \boldsymbol{\alpha} = \sum_{c=1}^{v} \boldsymbol{\alpha}^{(c)\top} \boldsymbol{Y}^{(c,c)} \big(\boldsymbol{F}^{(c,c)} \odot \boldsymbol{K}^{(c,c)}\big) \boldsymbol{Y}^{(c,c)} \boldsymbol{\alpha}^{(c)} ,
$$

and $\|\boldsymbol{F} - \mathbf{1}\mathbf{1}^\top\|_{\mathrm{F}}^2$ can be expressed as

$$
\|\boldsymbol{F} - \mathbf{1}\mathbf{1}^\top\|_{\mathrm{F}}^2 = \sum_{i,j=1}^{n} (\boldsymbol{F}_{ij} - 1)^2 = \sum_{c=1}^{v} \|\boldsymbol{F}^{(c,c)} - \mathbf{1}\mathbf{1}^\top\|_{\mathrm{F}}^2 + \mathrm{Const} ,
$$

where the constant is the sum of non-block-diagonal elements of $\bar{\boldsymbol{F}}$, and it does not affect the solution of Eq. (23). Specifically, the positive semi-definiteness on $\bar{\boldsymbol{F}}^{(c,c)}$ with $c = 1, 2, \ldots, v$ still guarantees that the whole matrix $\bar{\boldsymbol{F}}$ is PSD. Besides, the constraint in Eq. (23) is also decomposable, so the subproblems are separable and independent. As a result, the concatenation of their optimal solutions yields the optimal solution of Eq. (23). □

Based on the above lemma, we are ready to investigate the difference between $H(\boldsymbol{\alpha}^*, \boldsymbol{F}^*)$ and $H(\bar{\boldsymbol{\alpha}}, \bar{\boldsymbol{F}})$ as follows.

Theorem 3. *Assuming that* $0 < B_1 \leq \boldsymbol{F}_{ij} \leq B_2$*, with* $B = B_2 - B_1$*, and also introducing a partition indicator* $\{\pi(\boldsymbol{x}_1), \pi(\boldsymbol{x}_2), \ldots, \pi(\boldsymbol{x}_n)\}$*, we have*

$$\left|H(\boldsymbol{\alpha}^*, \boldsymbol{F}^*) - H(\bar{\boldsymbol{\alpha}}, \bar{\boldsymbol{F}})\right| \leq \frac{1}{2} BC^2 Q(\pi)\,,$$

with $Q(\pi) = \sum_{i,j:\pi(\boldsymbol{x}_i)\neq\pi(\boldsymbol{x}_j)}^{n} |\boldsymbol{K}(\boldsymbol{x}_i, \boldsymbol{x}_j)|$ *and the balance parameter* C*.*

Proof. We use $\bar{H}(\boldsymbol{\alpha}, \boldsymbol{F})$ to denote the objective function in Eq. (23) with the Gram matrix $\bar{\boldsymbol{K}}$. Hence, the optimal solution $(\bar{\boldsymbol{\alpha}}, \bar{\boldsymbol{F}})$ is a saddle point for the function $\bar{H}(\boldsymbol{\alpha}, \boldsymbol{F})$ due to the max-min problem in Eq. (23). It is easy to check $\bar{H}(\boldsymbol{\alpha}, \bar{\boldsymbol{F}}) \leq \bar{H}(\bar{\boldsymbol{\alpha}}, \bar{\boldsymbol{F}}) \leq \bar{H}(\bar{\boldsymbol{\alpha}}, \boldsymbol{F})$ for any feasible $\boldsymbol{\alpha}$ and $\boldsymbol{F}$.

Defining $H(\boldsymbol{\alpha}^*, \boldsymbol{F}^*)$ and $\bar{H}(\boldsymbol{\alpha}^*, \boldsymbol{F}^*)$, we can easily obtain

$$\bar{H}(\boldsymbol{\alpha}^*, \boldsymbol{F}^*) - H(\boldsymbol{\alpha}^*, \boldsymbol{F}^*) = \frac{1}{2} \sum_{i,j:\pi(\boldsymbol{x}_i)\neq\pi(\boldsymbol{x}_j)}^{n} \boldsymbol{\alpha}_i^* \boldsymbol{\alpha}_j^* y_i y_j \boldsymbol{F}_{ij}^* \boldsymbol{K}_{ij}\,, \tag{24}$$

with $\boldsymbol{F}_{ij}^* \triangleq \boldsymbol{F}^*(\boldsymbol{x}_i, \boldsymbol{x}_j)$. Similarly, we have

$$\bar{H}(\bar{\boldsymbol{\alpha}}, \boldsymbol{F}^*) - H(\bar{\boldsymbol{\alpha}}, \boldsymbol{F}^*) = \frac{1}{2} \sum_{i,j:\pi(\boldsymbol{x}_i)\neq\pi(\boldsymbol{x}_j)}^{n} \bar{\boldsymbol{\alpha}}_i \bar{\boldsymbol{\alpha}}_j y_i y_j \boldsymbol{F}_{ij}^* \boldsymbol{K}_{ij}\,.$$

Therefore, combining above equations, the upper bound of $H(\boldsymbol{\alpha}^*, \boldsymbol{F}^*) - H(\bar{\boldsymbol{\alpha}}, \bar{\boldsymbol{F}})$ can be derived by

$$\begin{aligned}
&H(\boldsymbol{\alpha}^*, \boldsymbol{F}^*) \leq \bar{H}(\bar{\boldsymbol{\alpha}}, \boldsymbol{F}^*) - \frac{1}{2} \sum_{i,j:\pi(\boldsymbol{x}_i)\neq\pi(\boldsymbol{x}_j)}^{n} \boldsymbol{\alpha}_i^* \boldsymbol{\alpha}_j^* y_i y_j \boldsymbol{F}_{ij}^* \boldsymbol{K}_{ij} \\
&= H(\bar{\boldsymbol{\alpha}}, \boldsymbol{F}^*) - \frac{1}{2} \sum_{i,j:\pi(\boldsymbol{x}_i)\neq\pi(\boldsymbol{x}_j)}^{n} \Big(\boldsymbol{\alpha}_i^* \boldsymbol{\alpha}_j^* - \bar{\boldsymbol{\alpha}}_i \bar{\boldsymbol{\alpha}}_j\Big) y_i y_j \boldsymbol{F}_{ij}^* \boldsymbol{K}_{ij} \\
&\leq H(\bar{\boldsymbol{\alpha}}, \bar{\boldsymbol{F}}) - \frac{1}{2} \sum_{i,j:\pi(\boldsymbol{x}_i)\neq\pi(\boldsymbol{x}_j)}^{n} \Big(\boldsymbol{\alpha}_i^* \boldsymbol{\alpha}_j^* - \bar{\boldsymbol{\alpha}}_i \bar{\boldsymbol{\alpha}}_j\Big) y_i y_j \boldsymbol{F}_{ij}^* \boldsymbol{K}_{ij} \\
&\leq H(\bar{\boldsymbol{\alpha}}, \bar{\boldsymbol{F}}) + \frac{1}{2} BC^2 Q(\pi)\,,
\end{aligned}$$

where the first inequality holds by $\bar{H}(\boldsymbol{\alpha}^*, \boldsymbol{F}^*) \leq \bar{H}(\bar{\boldsymbol{\alpha}}, \boldsymbol{F}^*)$ and Eq. (24). The second inequality admits by $H(\bar{\boldsymbol{\alpha}}, \boldsymbol{F}^*) \leq H(\bar{\boldsymbol{\alpha}}, \bar{\boldsymbol{F}})$. Similarly, $H(\boldsymbol{\alpha}^*, \boldsymbol{F}^*) - H(\bar{\boldsymbol{\alpha}}, \bar{\boldsymbol{F}})$ is lower bounded by

$$\begin{aligned}
&H(\boldsymbol{\alpha}^*, \boldsymbol{F}^*) = \bar{H}(\boldsymbol{\alpha}^*, \boldsymbol{F}^*) - \frac{1}{2} \sum_{i,j:\pi(\boldsymbol{x}_i)\neq\pi(\boldsymbol{x}_j)}^{n} \boldsymbol{\alpha}_i^* \boldsymbol{\alpha}_j^* y_i y_j \boldsymbol{F}_{ij}^* \boldsymbol{K}_{ij} \\
&\geq \bar{H}(\boldsymbol{\alpha}^*, \bar{\boldsymbol{F}}) - \frac{1}{2} \sum_{i,j:\pi(\boldsymbol{x}_i)\neq\pi(\boldsymbol{x}_j)}^{n} \boldsymbol{\alpha}_i^* \boldsymbol{\alpha}_j^* y_i y_j \boldsymbol{F}_{ij}^* \boldsymbol{K}_{ij} \\
&= \bar{H}(\boldsymbol{\alpha}^*, \bar{\boldsymbol{F}}) + \frac{1}{2} \sum_{i,j:\pi(\boldsymbol{x}_i)\neq\pi(\boldsymbol{x}_j)}^{n} \boldsymbol{\alpha}_i^* \boldsymbol{\alpha}_j^* y_i y_j (\bar{\boldsymbol{F}}_{ij} - \boldsymbol{F}_{ij}^*) \boldsymbol{K}_{ij} \\
&\geq H(\bar{\boldsymbol{\alpha}}, \bar{\boldsymbol{F}}) + \frac{1}{2} \sum_{i,j:\pi(\boldsymbol{x}_i)\neq\pi(\boldsymbol{x}_j)}^{n} \boldsymbol{\alpha}_i^* \boldsymbol{\alpha}_j^* y_i y_j (\bar{\boldsymbol{F}}_{ij} - \boldsymbol{F}_{ij}^*) \boldsymbol{K}_{ij} \\
&\geq H(\bar{\boldsymbol{\alpha}}, \bar{\boldsymbol{F}}) - \frac{1}{2} BC^2 Q(\pi)\,,
\end{aligned}$$

which concludes the proof that $\left|H(\boldsymbol{\alpha}^*, \boldsymbol{F}^*) - H(\bar{\boldsymbol{\alpha}}, \bar{\boldsymbol{F}})\right| \leq \frac{1}{2} BC^2 Q(\pi)$. □

Next we investigate the approximation error between the approximation solution $(\bar{\boldsymbol{\alpha}}, \bar{\boldsymbol{F}})$ and the optimal solution $(\boldsymbol{\alpha}^*, \boldsymbol{F}^*)$ by the following theorem.

Theorem 4. *Assuming that* $\bar{\boldsymbol{\alpha}} = \boldsymbol{\alpha}^* + \Delta\boldsymbol{\alpha}$*, and* $\bar{\boldsymbol{F}} = \boldsymbol{F}^* + \Delta\boldsymbol{F}$*, we then have*

$$\begin{cases}
\|\boldsymbol{\alpha}^* - \bar{\boldsymbol{\alpha}}\|_2^2 \leq \dfrac{B_2 C^2 Q(\pi)}{B_1 \|\boldsymbol{K}\|} + \dfrac{2BC^2}{B_1}\,, \\[2ex]
\|\boldsymbol{F}^* - \bar{\boldsymbol{F}}\|_{\mathrm{F}} \leq \dfrac{\|\boldsymbol{K}\|}{2\eta} \sqrt{\dfrac{nB_2 Q(\pi)}{B_1 \|\boldsymbol{K}\|} + \dfrac{2nB}{B_1}} C^2 + \dfrac{C^2 Q(\pi)}{4\eta}\,.
\end{cases}$$

Proof. We firstly derive the error bound with respect to $\boldsymbol{\alpha}$, and then bound $\|\boldsymbol{F}^* - \bar{\boldsymbol{F}}\|_{\mathrm{F}}$. Only considering $\boldsymbol{\alpha}$, we use an equivalent form of Eq. (21), that is

$$\begin{aligned}
&\max_{\boldsymbol{\alpha}} \min_{\boldsymbol{F}\in\mathcal{S}_+^n} \mathbf{1}^\top \boldsymbol{\alpha} - \frac{1}{2} \boldsymbol{\alpha}^\top \boldsymbol{Y}(\boldsymbol{F} \odot \boldsymbol{K}) \boldsymbol{Y} \boldsymbol{\alpha} \\
&\text{s.t. } \mathbf{0} \leq \boldsymbol{\alpha} \leq C\mathbf{1},\ \|\boldsymbol{F} - \mathbf{1}\mathbf{1}^\top\|_{\mathrm{F}}^2 \leq R^2\,,
\end{aligned} \tag{25}$$

which is demonstrated by Eq. (4). The optimality condition of $\boldsymbol{\alpha}$ in Eq. (25) is

$$\Big(\nabla_{\boldsymbol{\alpha}} H(\boldsymbol{\alpha}^*, \boldsymbol{F}^*)\Big)_i \begin{cases} = 0 & \text{if } 0 < \boldsymbol{\alpha}_i^* < C, \\ \leq 0 & \text{if } \boldsymbol{\alpha}_i^* = 0, \\ \geq 0 & \text{if } \boldsymbol{\alpha}_i^* = C, \end{cases} \tag{26}$$

where $\nabla_{\boldsymbol{\alpha}} H(\boldsymbol{\alpha}^*, \boldsymbol{F}^*) = \mathbf{1} - \boldsymbol{Y}(\boldsymbol{F}^* \odot \boldsymbol{K})\boldsymbol{Y}\boldsymbol{\alpha}^*$. Since $\bar{\boldsymbol{\alpha}}$ is a feasible solution satisfying $0 \leq \bar{\boldsymbol{\alpha}}_i \leq C$, we have $(\Delta\boldsymbol{\alpha})_i \geq 0$ if $\boldsymbol{\alpha}_i^* = 0$ and $(\Delta\boldsymbol{\alpha})_i \leq 0$ if $\boldsymbol{\alpha}_i^* = C$. Accordingly, the following inequality holds

$$(\Delta\boldsymbol{\alpha})^\top (\mathbf{1} - \boldsymbol{Y}(\boldsymbol{F}^* \odot \boldsymbol{K})\boldsymbol{Y}\boldsymbol{\alpha}^*) = \sum_{i=1}^{n} (\Delta\boldsymbol{\alpha})_i \big(\nabla_{\boldsymbol{\alpha}} H(\boldsymbol{\alpha}^*, \boldsymbol{F}^*)\big)_i \leq 0. \tag{27}$$

Next, according to Eq. (27), $H(\bar{\boldsymbol{\alpha}}, \bar{\boldsymbol{F}})$ in Eq. (25) can be decomposed into

$$\begin{aligned}
H(\bar{\boldsymbol{\alpha}}, \bar{\boldsymbol{F}}) &= H(\boldsymbol{\alpha}^*, \boldsymbol{F}^*) - \frac{1}{2} (\Delta\boldsymbol{\alpha})^\top \boldsymbol{Y}(\bar{\boldsymbol{F}} \odot \boldsymbol{K})\boldsymbol{Y}(\Delta\boldsymbol{\alpha}) \\
&\quad + \mathbf{1}^\top (\Delta\boldsymbol{\alpha}) - \boldsymbol{\alpha}^* \boldsymbol{Y}(\boldsymbol{F}^* \odot \boldsymbol{K})\boldsymbol{Y}(\Delta\boldsymbol{\alpha}) \\
&\quad - \frac{1}{2} (\boldsymbol{\alpha}^*)^\top \boldsymbol{Y}(\Delta\boldsymbol{F} \odot \boldsymbol{K})\boldsymbol{Y}\boldsymbol{\alpha}^* - (\boldsymbol{\alpha}^*)^\top \boldsymbol{Y}(\Delta\boldsymbol{F} \odot \boldsymbol{K})\boldsymbol{Y}\Delta\boldsymbol{\alpha} \\
&\leq H(\boldsymbol{\alpha}^*, \boldsymbol{F}^*) - \frac{1}{2} (\Delta\boldsymbol{\alpha})^\top \boldsymbol{Y}(\bar{\boldsymbol{F}} \odot \boldsymbol{K})\boldsymbol{Y}(\Delta\boldsymbol{\alpha}) \\
&\quad - (\boldsymbol{\alpha}^*)^\top \boldsymbol{Y}(\Delta\boldsymbol{F} \odot \boldsymbol{K})\boldsymbol{Y}\Delta\boldsymbol{\alpha}\,.
\end{aligned}$$

Further, the above inequality indicates that

$$\begin{aligned}
\frac{1}{2} (\Delta\boldsymbol{\alpha})^\top \boldsymbol{Y}(\boldsymbol{F}^* \odot \boldsymbol{K})\boldsymbol{Y}(\Delta\boldsymbol{\alpha}) &\leq H(\boldsymbol{\alpha}^*, \boldsymbol{F}^*) - H(\bar{\boldsymbol{\alpha}}, \bar{\boldsymbol{F}}) \\
&\quad - \frac{1}{2} (\Delta\boldsymbol{\alpha})^\top \boldsymbol{Y}(\bar{\boldsymbol{F}} \odot \boldsymbol{K})\boldsymbol{Y}(\Delta\boldsymbol{\alpha}) \\
&\leq \frac{1}{2} B_2 C^2 Q(\pi) + BC^2 \|\boldsymbol{K}\|\,,
\end{aligned}$$

which concludes the bound

$$\|\boldsymbol{\alpha}^* - \bar{\boldsymbol{\alpha}}\|_2^2 \leq \frac{B_2 C^2 Q(\pi)}{B_1 \|\boldsymbol{K}\|} + \frac{2BC^2}{B_1}\,. \tag{28}$$

Next our target is to bound $\|\boldsymbol{F}^* - \bar{\boldsymbol{F}}\|_{\mathrm{F}}$. Defining

$$\boldsymbol{F}^*(\boldsymbol{\alpha}^*) = \mathcal{J}_{\frac{\tau}{2}}\Big(\mathbf{1}\mathbf{1}^\top + \underbrace{\frac{1}{4\eta}\operatorname{diag}((\boldsymbol{\alpha}^*)^\top \boldsymbol{Y})\boldsymbol{K}\operatorname{diag}((\boldsymbol{\alpha}^*)^\top \boldsymbol{Y})}_{\triangleq \boldsymbol{\Gamma}^*(\boldsymbol{\alpha}^*)}\Big),$$

$$\bar{\boldsymbol{F}}(\bar{\boldsymbol{\alpha}}) = \mathcal{J}_{\frac{\tau}{2}}\Big(\mathbf{1}\mathbf{1}^\top + \underbrace{\frac{1}{4\eta}\operatorname{diag}(\bar{\boldsymbol{\alpha}}^\top \boldsymbol{Y})\bar{\boldsymbol{K}}\operatorname{diag}(\bar{\boldsymbol{\alpha}}^\top \boldsymbol{Y})}_{\triangleq \bar{\boldsymbol{\Gamma}}(\bar{\boldsymbol{\alpha}})}\Big),$$

we have

$$\begin{aligned}\big\|\boldsymbol{F}^*(\boldsymbol{\alpha}^*) - \bar{\boldsymbol{F}}(\bar{\boldsymbol{\alpha}})\big\|_{\mathrm{F}} &\le \big\|\boldsymbol{\Gamma}^*(\boldsymbol{\alpha}^*) - \bar{\boldsymbol{\Gamma}}(\bar{\boldsymbol{\alpha}})\big\|_{\mathrm{F}} \\ &\le \big\|\boldsymbol{\Gamma}^*(\boldsymbol{\alpha}^*) - \boldsymbol{\Gamma}^*(\bar{\boldsymbol{\alpha}})\big\|_{\mathrm{F}} + \big\|\boldsymbol{\Gamma}^*(\bar{\boldsymbol{\alpha}}) - \bar{\boldsymbol{\Gamma}}(\bar{\boldsymbol{\alpha}})\big\|_{\mathrm{F}},\end{aligned} \tag{29}$$

with $\boldsymbol{\Gamma}^*(\bar{\boldsymbol{\alpha}}) = \frac{1}{4\eta}\operatorname{diag}(\bar{\boldsymbol{\alpha}}^\top \boldsymbol{Y})\boldsymbol{K}\operatorname{diag}(\bar{\boldsymbol{\alpha}}^\top \boldsymbol{Y})$. The first term in Eq. (29) can be bounded by Lemma 3. Combining it with the derived bound regarding to $\|\boldsymbol{\alpha}^* - \bar{\boldsymbol{\alpha}}\|_2$ in Eq. (28), we have

$$\begin{aligned}\big\|\boldsymbol{\Gamma}^*(\boldsymbol{\alpha}^*) - \boldsymbol{\Gamma}^*(\bar{\boldsymbol{\alpha}})\big\|_{\mathrm{F}} &\le \frac{\|\boldsymbol{K}\|_{\mathrm{F}}}{4\eta}\big\|\boldsymbol{\alpha}^* + \bar{\boldsymbol{\alpha}}\big\|_2\big\|\boldsymbol{\alpha}^* - \bar{\boldsymbol{\alpha}}\big\|_2 \\ &\le \frac{\|\boldsymbol{K}\|}{2\eta}\sqrt{\frac{nB_2Q(\pi)}{B_1\|\boldsymbol{K}\|} + \frac{2nB}{B_1}}C^2.\end{aligned} \tag{30}$$

Next we bound the second term in Eq. (29) as

$$\begin{aligned}\big\|\boldsymbol{\Gamma}^*(\bar{\boldsymbol{\alpha}}) - \bar{\boldsymbol{\Gamma}}(\bar{\boldsymbol{\alpha}})\big\|_{\mathrm{F}} &= \frac{1}{4\eta}\operatorname{diag}(\bar{\boldsymbol{\alpha}}^\top \boldsymbol{Y})(\boldsymbol{K} - \bar{\boldsymbol{K}})\operatorname{diag}(\bar{\boldsymbol{\alpha}}^\top \boldsymbol{Y}) \\ &\le \frac{1}{4\eta}\sum_{i,j:\pi(\boldsymbol{x}_i)\neq\pi(\boldsymbol{x}_j)}^{n}\bar{\alpha}_i\bar{\alpha}_i y_i y_j \boldsymbol{K}_{ij} \\ &\le \frac{1}{4\eta}C^2Q(\pi).\end{aligned} \tag{31}$$

Combining Eqs. (29), (30), and (31), we conclude the proof. □

Remark: Compared with the exact bound $\|\boldsymbol{\alpha}^* - \bar{\boldsymbol{\alpha}}\|_2^2 \le nC^2$, the derived approximation error about $\boldsymbol{\alpha}$ by Theorem 4 is independent of n. Compared to the intuitive bound $\|\boldsymbol{F}^* - \bar{\boldsymbol{F}}\|_{\mathrm{F}} \le nB$, the obtained error bound regarding to $\boldsymbol{F}$ by Theorem 4 is $\mathcal{O}(\sqrt{n})$.

The above theoretical results demonstrate the approximation performance of the subproblems to the whole problem, regarding to the difference of their respective objective function values in Theorem 3, and the difference of their respective optimization variables in Theorem 4. Besides, in SVM, we also concern about the relationship of support/non-support vectors between the subproblems and the whole problem. Accordingly, we present the following theorem to explain this issue.

Theorem 5. *Assuming that $\boldsymbol{x}_i$ is not a support vector of the subproblem, i.e., $\bar{\boldsymbol{\alpha}}_i = 0$, $\boldsymbol{x}_i$ will also not be a support vector of the whole problem i.e., $\boldsymbol{\alpha}_i = 0$, under the following condition*

$$\Big(\nabla_{\boldsymbol{\alpha}}\bar{H}(\bar{\boldsymbol{\alpha}}, \bar{\boldsymbol{F}})\Big)_i \le -\Big((B + 2B_2)CQ(\pi) + (B + B_2)\boldsymbol{K}_{\max}C\Big),$$

where $\boldsymbol{K}_{\max}$ satisfies $\boldsymbol{K}_{ij} \le \boldsymbol{K}_{\max}$ for any $i, j = 1, 2, \ldots, n$.

Proof. We decompose $\nabla_{\boldsymbol{\alpha}}H(\boldsymbol{\alpha}^*, \boldsymbol{F}^*)$ into

$$\begin{aligned}\Big(\nabla_{\boldsymbol{\alpha}}H(\boldsymbol{\alpha}^*, \boldsymbol{F}^*)\Big)_i &= \Big(\nabla_{\boldsymbol{\alpha}}\bar{H}(\bar{\boldsymbol{\alpha}}, \bar{\boldsymbol{F}})\Big)_i - \Big(\boldsymbol{Y}(\boldsymbol{F}^* \odot \boldsymbol{K})\boldsymbol{Y}\Delta\boldsymbol{\alpha}\Big)_i \\ &\quad - \Big(\boldsymbol{Y}(\Delta\boldsymbol{F} \odot \boldsymbol{K})\boldsymbol{Y}\boldsymbol{\alpha}^*\Big)_i + \sum_{j:\pi(\boldsymbol{x}_i)\neq\pi(\boldsymbol{x}_j)}^{n}\bar{\boldsymbol{\alpha}}_j y_i y_j \boldsymbol{F}_{ij}^*\boldsymbol{K}_{ij} \\ &\quad + \sum_{j:\pi(\boldsymbol{x}_i)\neq\pi(\boldsymbol{x}_j)}^{n}\boldsymbol{\alpha}_j^* y_i y_j (\Delta\boldsymbol{F})_{ij}\boldsymbol{K}_{ij} \\ &\le \Big(\nabla_{\boldsymbol{\alpha}}\bar{H}(\bar{\boldsymbol{\alpha}}, \bar{\boldsymbol{F}})\Big)_i + (B_2 - B_1)CQ(\pi) + 2B_2CQ(\pi) \\ &\quad + (B_2 - B_1)\boldsymbol{K}_{\max}C + B_2\boldsymbol{K}_{\max}C \\ &\le \Big(\nabla_{\boldsymbol{\alpha}}\bar{H}(\bar{\boldsymbol{\alpha}}, \bar{\boldsymbol{F}})\Big)_i + (B + 2B_2)CQ(\pi) + (B + B_2)\boldsymbol{K}_{\max}C,\end{aligned}$$

which implies $\big(\nabla_{\boldsymbol{\alpha}}H(\boldsymbol{\alpha}^*, \boldsymbol{F}^*)\big)_i \le 0$ when $\bar{\boldsymbol{\alpha}}_i = 0$. As a result, we can conclude that $\boldsymbol{\alpha}_i = 0$ from the optimality condition of problem (21) in Eq. (26). □

6 Experimental Results

This section evaluates the performance of our DANK model with several representative kernel learning algorithms on classification and regression benchmark datasets. All the experiments implemented in MATLAB are conducted on a workshop with an Intel® Xeon® E5-2695 CPU (2.30 GHz) and 64GB RAM.

6.1 Classification tasks

We conduct experiments on the UCI Machine Learning Repository[2], and *CIFAR-10* database[3] for image classification. Besides, we evaluate the kernel approximation performance on two large datasets including *ijcnn1* and *covtype*[4].

6.1.1 Classification Results on UCI database

Ten datasets from the UCI database are used to evaluate our DANK model embedded in SVM. Here we describe experimental settings and the compared algorithms as follows.

Experimental Settings: Table 1 lists a brief description of these ten datasets including the number of training data n and the feature dimension d. After normalizing the data to $[0, 1]^d$ in advance, we randomly pick half of the data for training and the rest for test except for *monks1*, *monks2*, and *monks3*. In these three datasets, both training and test data have been provided. The Gaussian kernel $k(\boldsymbol{x}_i, \boldsymbol{x}_j) = \exp(-\|\boldsymbol{x}_i - \boldsymbol{x}_j\|^2/\sigma^2)$ is chosen as the initial kernel in our model. The kernel width σ and the balance parameter C are tuned by 5-fold cross validation on a grid of points, i.e., $\sigma = [2^{-5}, 2^{-4}, \ldots, 2^5]$ and $C = [2^{-5}, 2^{-4}, \ldots, 2^5]$. We experimentally set the penalty parameter τ to 0.01. The regularization parameter η is fixed to $\|\boldsymbol{\alpha}\|^2$ obtained by SVM. The experiments are conducted 10 times on these ten datasets.

Compared Methods: We include the following kernel learning based algorithms:

- BMKL [44]: A multiple kernel learning algorithm uses Bayesian approach to ensemble the Gaussian kernels with ten different kernel widths and the polynomial kernels with three different degrees.

2. https://archive.ics.uci.edu/ml/datasets.html
3. https://www.cs.toronto.edu/~kriz/cifar.html
4. Both data sets are available at https://www.csie.ntu.edu.tw/~cjlin/libsvmtools/datasets/

TABLE 1
Comparison results in terms of classification accuracy (mean±std. deviation %) on the UCI datasets. The best performance is highlighted in **bold**. The classification accuracy on the training data is presented by *italic*, and does not participate in ranking.

Dataset	(d, n)	DMKL [43]	BMKL [44]	RF [17]	SVM-CV		KNPL [24]	DANK	
		Test	Test	Test	Training	Test	Test	Training	Test
diabetic	(19, 1151)	72.95±1.03	74.97±0.49	72.38±0.89	*80.71±3.98*	73.00±1.74	**81.98**±1.72	*87.04±1.91*	80.63±1.78
heart	(13, 270)	80.29±2.70	87.33±0.23	79.11±2.42	*88.96±3.07*	81.92±2.47	87.47±3.90	*94.37±1.72*	**87.88**±2.89
monks1	(6, 124)	84.12±3.62	78.91±2.55	**84.44**±0.96	*90.32±0.00*	81.48±0.00	83.38±3.35	*100.0±0.00*	83.51±1.57
monks2	(6, 169)	77.24±5.72	82.12±1.31	73.61±1.19	*100.0±0.00*	85.81±1.40	83.33±1.68	*100.0±0.00*	**86.68**±0.91
monks3	(6, 122)	90.69±3.20	**94.00**±1.09	93.75±0.67	*96.22±1.50*	93.07±1.24	88.78±1.23	*97.21±1.88*	93.19±0.94
sonar	(60, 208)	80.57±5.06	84.80±0.60	80.57±3.14	*99.90±0.30*	85.36±3.17	85.86±2.86	*100.0±0.00*	**87.11**±2.93
spect	(21, 80)	78.75±4.47	78.88±0.84	76.04±2.76	*87.00±3.78*	73.10±3.23	**79.73**±4.82	*93.25±7.10*	78.60±4.42
glass	(9,214)	72.82±2.46	68.22±4.67	68.27±2.25	*77.10±6.94*	69.53±4.83	72.46±2.31	*89.71±6.16*	**73.90**±1.54
fertility	(9,100)	80.40±4.77	84.40±1.62	84.40±4.33	*94.40±5.36*	85.20±1.78	85.60±3.84	*97.30±3.34*	**86.40**±5.36
wine	(13,178)	95.00±3.13	95.06±2.87	95.11±1.18	*99.55±1.00*	94.77±1.52	**96.17**±2.04	*99.55±1.00*	**96.17**±2.19

- DMKL [43]: A three-layer kernel framework utilizes four unique base kernels including a linear kernel, a Gaussian kernel, a sigmoid kernel, and a polynomial kernel.
- RF [17]: A nonparametric kernel learning framework creates randomized features, and then solves a simple optimization problem to select a subset. Finally, the kernel is learned from the optimized features by target alignment.
- KNPL [24]: As the conference version of the proposed DANK model, it does not consider the bounded constraint and the low rank structure on $\boldsymbol{F}$, and utilizes an alternating iterative algorithm to solve the corresponding problem.
- SVM-CV: The SVM classifier with cross validation is served as a baseline.

Experimental Results: Table 1 reports average classification accuracy and the standard deviation of each compared algorithm on the test data. Specifically, we also present the classification accuracy of our DANK model and SVM-CV on the training data to show their respective model flexibilities.

Compared with the baseline SVM-CV, the proposed DANK model significantly improves its flexibility on *diabetic*, *heart*, *monks1*, *spect*, and *glass* in terms of the training accuracy. Our DANK model is able to capture different local statistics of training data, and accordingly achieves noticeable improvements on test data. On the remaining datasets, the training accuracy yielded by SVM-CV indicates that the model flexibility is enough. In this case, our DANK method can hardly achieve a huge promotion on these datasets, and the performance margins are about 0%∼2%. Besides, compared with other representative kernel based algorithms including DMKL, BMKL, and RF, our methods including KNPL and DANK yield favorable performance.

In general, the improvements on the classification accuracy demonstrate the effectiveness of the learned adaptive matrix which conveys richer information than other kernel methods. Thereby, our model has good adaptivity to the training and test data.

6.1.2 Results on CIFAR-10 dataset

In this section, we test our model on a representative dataset *CIFAR-10* [45] for natural image classification task. This dataset contains 60,000 color images with the size of $32 \times 32 \times 3$ in 10 categories, of which 50,000 images are used for training and the rest are for testing. Specifically, each color image is represented by the feature extracted from a convolutional neural network. In our experiment,

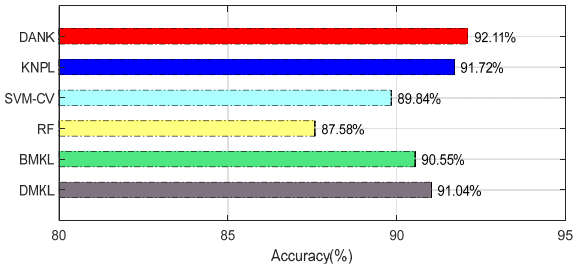


Fig. 3. Performance of the compared algorithms on *CIFAR-10* dataset.

TABLE 2
Dataset statistics and parameter settings on two large datasets.

datasets	d	#training	#test	C	$1/\sigma^2$	#clusters
ijcnn1	22	49,990	91,701	32	2	50
covtype	54	464,810	116,202	32	32	200

we use the features extracted by VGG16 with batch normalization [46], which is trained with 240 epochs and a min-batch size of 64. The learning rate starts from 0.1 and then is divided by 10 at 120-th, 160-th, and 200-th epoch. After that, for each image, a 4096 dimensional feature vector is obtained according to the output of the first fully-connected layer in this neural network.

The test accuracy of the compared algorithms is shown in Fig. 3. We can see that, our two methods (KNPL and DANK) achieve promising classification accuracy with 91.72% and 92.11%, respectively. Specifically, our DANK model outperforms SVM-CV with an accuracy margin of 2.27%. Such improvement over SVM-CV on the test accuracy demonstrates that our methods equipped with the introduced kernel adjustment strategy are able to enhance the model flexibility, and thus achieving good performance.

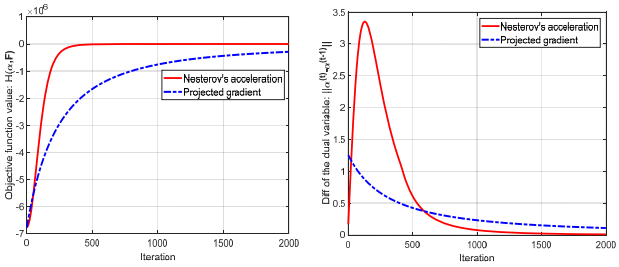
6.1.3 Results on large-scale datasets

Two large datasets including *ijcnn1* and *covtype* are used to evaluate the kernel approximation performance. Table 2 reports the dataset statistics (*i.e.*, the feature dimension d, the number of training examples, and the number of test data) and parameter settings including the balance parameter C, the kernel width σ, and the number of clusters. Table 3 presents the test accuracy and training time of various compared algorithms including DMKL, BMKL,

TABLE 3
Comparison of test accuracy and training time of all the compared algorithms on *ijcnn* and *covetype*.

Method		SVM-SMO	DMKL		BMKL		RF	DANK	
Dataset	Setting	exact	exact	scalable	exact	scalable	exact	exact	scalable
ijcnn	acc.(%)	96.58	96.75	96.39	98.52	97.77	93.05	98.84	98.23
	time(sec.)	112.47	9122.8	94.23	109967	1916.9	25.06	28548	553.84
covtype	acc.(%)	96.15	×[1]	94.91	×	91.27	79.15	×	96.89
	time(sec.)	3972.5	×	4663.4	×	94632	364.5	×	7159.6

[1] These methods try to directly solve the optimization problem on *covetype* but fail due to the memory limit.



(a) (b)

Fig. 4. Comparison between projected gradient method and our Nesterov's acceleration on *heart* dataset. The X-axis is the number of iterations. The Y-axis is the objective function value $H(\boldsymbol{\alpha}, \boldsymbol{F})$ in (a) and the dual variable difference $\|\boldsymbol{\alpha}^{(t)} - \boldsymbol{\alpha}^{(t-1)}\|_2$ in (b).

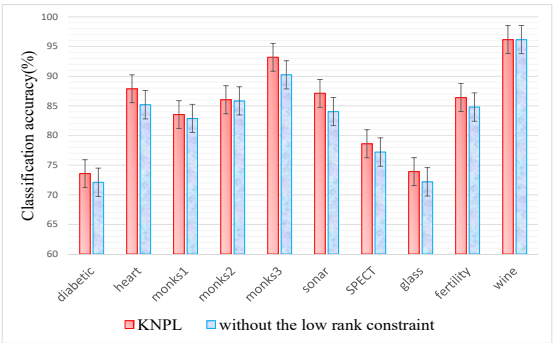


Fig. 5. Influence of the low rank constraint on test classification accuracy.

our DANK method, SVM-SMO [47] (the cache is set to 5000) and RF conducted in the following two settings.

In the first setting ("exact"), we attempt to directly test these algorithms over the entire training data. Experimental results indicate that, without the decomposition-based scalable approach, our DANK method achieves the best test accuracy with 98.84% on *ijcnn*. However, under this setting, DMKL, BMKL, and our method are infeasible to deal with a quite large dataset *covetype* due to the memory limit. Differently, SVM-SMO and RF can be directly used for large-scale datasets.

In the second setting ("scalable"), we incorporate the kernel approximation scheme into DMKL, BMKL, and DANK, and evaluate them on *ijcnn1* and *covtype*. Experimental results show that, by such decomposition-based scalable approach, these three methods are scalable on *ijcnn* and *covetype*, respectively. When compared with the direct solution of the optimization problem in the "exact" setting, DMKL, BMKL, and DANK equipped with kernel approximation speed up about 100x, 50x, and 50x on *ijcnn*, respectively. Specifically, on these two datasets, our DANK method using the approximation scheme still performs better than SVM-SMO on the test accuracy, which demonstrates the effectiveness of the proposed flexible kernel learning framework.

From the results in above two settings, we see that our DANK method achieves promising test accuracy no matter whether the kernel approximation scheme is incorporated or not. What's more, such approximation scheme makes BMKL, DMKL, and our DANK method feasible to large datasets with huge promotion in terms of computational efficiency.

6.1.4 Convergence experiments

Here we investigate the convergence of the used first-order Nesterov's smooth optimization method on *heart* dataset. The standard projected gradient method is served as a baseline.

In Fig. 4(a), we plot the objective function value $H(\boldsymbol{\alpha}, \boldsymbol{F})$ versus iteration by the standard projected gradient method (in blue dashed line) and its Nesterov's acceleration (in red solid line), respectively. One can see that the developed first-order Nesterov's smooth optimization method converges faster than the projected gradient method, so the feasibility of employing Nesterov's acceleration for solving problem (5) is verified. Besides, to illustrate the convergence of $\{\boldsymbol{\alpha}^{(t)}\}_{t=0}^{\infty}$, we plot $\|\boldsymbol{\alpha}^{(t)} - \boldsymbol{\alpha}^{(t-1)}\|_2$ versus iteration in Fig. 4(b). We find that the sequence $\{\boldsymbol{\alpha}^{(t)}\}_{t=0}^{\infty}$ yielded by the Nesterov's acceleration algorithm significantly decays in the first 500 iterations, which leads to quick convergence to an optimal solution. Hence, compared with projected gradient method, the Nesterov's smooth optimization method is able to efficiently solve the targeted convex optimization problem in this paper.

6.1.5 Ablation study on the low rank constraint

In large-scale case, we do not consider the low rank regularizer on $\boldsymbol{F}$ due to its inseparable property for efficient optimization. Here we experimentally specialize in the influence of $\|\boldsymbol{F}\|_*$ on the test classification accuracy in both small and large-scale datasets.

For small-scale situations, we choose ten datasets from the UCI database appeared in Section 6.1.1 to verify the effectiveness of $\|\boldsymbol{F}\|_*$. Fig. 5 shows that, in terms of the test accuracy, apart from *monks1*, *monks2* and *wine* datasets, our method without the low rank regularizer loses about 2% accuracy on the remaining datasets. It indicates that the low rank constraint is important in small datasets and here we attempt to explain this issue. Without the low rank constraint, each entry in the learned adaptive matrix $\boldsymbol{F}$ can arbitrarily vary in the solution space $\mathcal{F}$. As a result, our model has the $\mathcal{O}(n^2)$ capability of "scattering" n training data, which would lead to over-fitting. Besides, the learned $\boldsymbol{F}$ might be sophisticated, therefore, it is not easily extended to $\boldsymbol{F}'$ for test data by the simple nearest neighbor scheme.

For large-scale situations, Table 3 reports that the exact solution of our DANK model achieves the test accuracy with 98.84%. In contrast, after omitting the low rank regularizer $\|\boldsymbol{F}\|_*$ and using the decomposition-based scalable approach, the test accuracy of our DANK method in "scalable" setting is 98.23%. Such slight decrease on the classification accuracy indicates that the dropping of $\|\boldsymbol{F}\|_*$ in large datasets is reasonable and acceptable.

From above observations and analyses, we conclude that the low rank constraint in our model is important for small-scale cases, but can be dropped in large datasets due to the neglectable decrease on the test accuracy. The reason behind this might be that the kernel in large datasets often inherits the rapid decaying spectra [39], so the explicit regularizer on $\boldsymbol{F}$ will not have significance influence on the final results.

6.2 Regression

This section focuses on the proposed DANK model embedded in SVR for regression tasks. We firstly conduct the experiments on several synthetic datasets, to examine the performance of our method on recovering 1-D and 2-D test functions. After that, eight regression datasets[5] are used to test our model and other representative regression algorithms. The used evaluation metric here is relative mean square error (RMSE) between the learned regression function $\hat{g}(\boldsymbol{x})$ and the target label $\boldsymbol{y}$ over n data points

$$\text{RMSE} = \frac{\sum_{i=1}^{n}\left(\hat{g}(\boldsymbol{x}_i) - y_i\right)^2}{\sum_{i=1}^{n}\left(y_i - \mathbb{E}(\boldsymbol{y})\right)^2}\,.$$

6.2.1 Synthetic Data

Here we test the approximation performance of our method on 1-D and 2-D test functions. The SVR with Gaussian kernel is served as a baseline. The representative 1-D step function is defined by

$$g(s,w,a,x) = \left(\frac{\tanh\left(\frac{ax}{w} - a\lfloor\frac{x}{w}\rfloor - \frac{a}{2}\right)}{2\tanh\left(\frac{a}{2}\right)} + \frac{1}{2} + \left\lfloor\frac{x}{w}\right\rfloor\right)s\,,$$

where s is the step hight, w is the period, and a controls the smoothness of the function g. In our experiment, s, w and a are set to 3, 2 and 0.05, respectively. We plot the step function on $[-5, 5]$ as shown in Fig. 6(a). One can see that the approximation function generated by SVR-CV (blue dashed line) yields a larger deviation than that of our DANK model (red solid line). The quantitative analysis is also reported here, *i.e.*, the RMSE of SVR is 0.013, while our DANK model achieves a promising approximation error with a value of 0.004.

Apart from the 1-D function, we use a 2-D test function to test SVR-CV and the proposed DANK model. The 2-D test function [49] $g(u,v) \in [-0.5, 0.5] \times [-0.5, 0.5]$ is established as

$$g(u,v) = 42.659\Big(0.1 + (u-0.5)\big(g_1(u,v) + 0.05\big)\Big)\,,$$

where $g_1(u,v)$ is defined by

$$g_1(u,v) = (u-0.5)^4 - 10(u-0.5)^2(v-0.5)^2 + 5(v-0.5)^4\,.$$

We uniformly sample 400 data points by $g(u,v)$ as shown in Fig. 6(b), and then use SVR-CV and DANK to learn a regression function from the sampled data. The regression results by SVR-CV and our DANK model are shown in Fig. 6(c) and Fig. 6(d), respectively. Intuitively, when we focus on the upwarp of the original function, our method is more similar to the test function than SVR-CV. In terms of RMSE, the error of our method for regressing the test function is 0.007, which is more precise than that of SVR-CV with 0.042.

5. The compared datasets are available at http://www.csie.ntu.edu.tw/~cjlin/libsvmtools/datasets/binary.html

6.2.2 Regression Results on UCI datasets

For real-world situations, we compare the proposed DANK model with other representative regression algorithms on eight datasets from the UCI database. In [44], the authors extend BMKL to regression tasks, and then we include it for comparisons. Apart from SVR-CV and BMKL, the Nadaraya-Watson (NW) estimator with metric learning [48] is also taken into comparison. The remaining experimental settings follow with the classification tasks on the UCI database illustrated in Section 6.1.1.

Table 4 lists a brief statistics of these eight datasets, and reports the average prediction accuracy and standard deviation of every compared algorithm. Specifically, we also present the regression performance of our DANK model and SVR-CV on the training data to show their respective model flexibilities. From the results on four datasets including *bodyfat*, *pyrim*, *space*, and *mpg*, we observe that our method statistically achieves the best prediction performance. On *triazines*, *housing*, and *mg* datasets, the prediction result of our method is not the best among all the comparators. However, the proposed DANK model still shows more encouraging performance than SVR-CV in terms of the RMSE on the training and test data.

7 Conclusion and Future Work

In this work, an effective data-adaptive strategy is investigated to enhance the model flexibility for nonparametric kernel learning based algorithms. Each entry in the Gram matrix can be carefully and flexibly learned from the data, leading to an improved data-adaptive kernel. As a result of such data-driven scheme, the proposed DANK model embedded in SVM and SVR shows significant flexibility to adapt to data. The applicability of our DANK model for classification and regression tasks is demonstrated by the experiments on synthetic and real datasets. In addition, we develop a decomposed-based scalable approach to make our DANK model feasible to large datasets, of which the effectiveness has been verified by both experimental results and theoretical demonstration. However, there is an interesting question left unanswered regarding the out-of-sample extensions issue. Albeit effective, the used nearest neighbor scheme may lead to the inconsistency between training kernel and test kernel. Some sophisticated out-of-sample extension based algorithms [27], [29] can be further exploited to nonparametric kernel learning.

References

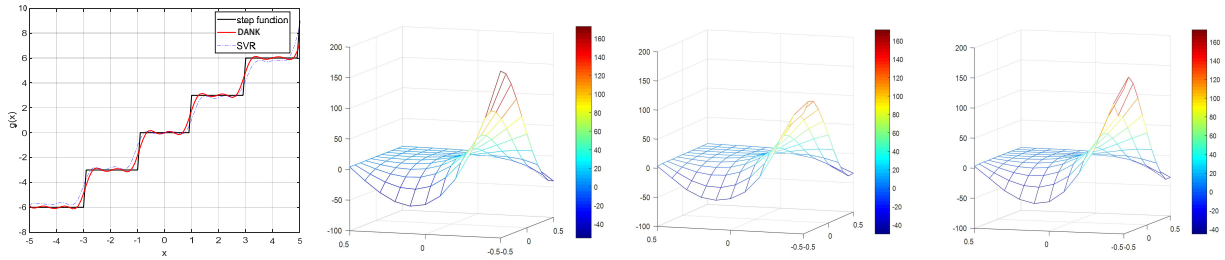
[1] Bernhard Schölkopf and Alexander J Smola, *Learning with kernels: Support Vector Machines, Regularization, Optimization, and Beyond*, MIT Press, 2003.

[2] Mehran Kafai and Kave Eshghi, "CROification: accurate kernel classification with the efficiency of sparse linear SVM," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 2018.

[3] Vladimir N. Vapnik, *The Nature of Statistical Learning Theory*, Springer, 1995.

[4] Harris Drucker, Christopher JC Burges, Linda Kaufman, Alex J Smola, and Vladimir Vapnik, "Support vector regression machines," in *Proceedings of Advances in neural information processing systems*, 1997, pp. 155–161.

[5] Carl Edward Rasmussen, "Gaussian processes in machine learning," in *Advanced lectures on machine learning*, pp. 63–71. Springer, 2004.



(a) recovering a step function (b) a 2-D function (c) recovering by SVR-CV (d) recovering by DANK

Fig. 6. Approximation for 1-D and 2-D functions is displayed by SVR-CV and our DANK model.

TABLE 4
Comparison results of various methods on UCI datasets in terms of RMSE (mean±std. deviation %). The best performance is highlighted in **bold**. The RMSE on the training data is presented by *italic*, and does not participate in ranking.

Dataset	(d, n)	BMKL [44]	NW [48]	SVR-CV		DANK	
		Test	Test	Training	Test	Training	Test
bodyfat	(14, 252)	0.101±0.065	0.097±0.010	*0.007±0.000*	0.101±0.002	*0.008±0.000*	**0.094**±0.017
pyrim	(27, 74)	0.514±0.223	0.604±0.115	*0.013±0.010*	0.791±0.432	*0.007±0.004*	**0.475**±0.191
space	(6, 3107)	0.249±0.160	0.308±0.004	*0.226±0.144*	0.258±0.134	*0.106±0.067*	**0.221**±0.176
triazines	(60, 186)	**0.725**±0.124	0.885±0.039	*0.113±0.084*	0.815±0.206	*0.009±0.012*	0.743±0.147
cpusmall	(12, 8912)	0.133±0.004	**0.037**±0.009	*0.037±0.004*	0.104±0.002	*0.001±0.000*	0.128±0.002
housing	(13, 506)	0.286±0.027	**0.177**±0.015	*0.085±0.049*	0.267±0.023	*0.069±0.013*	0.210±0.017
mg	(6, 1385)	0.297±0.013	**0.294**±0.010	*0.163±0.076*	0.407±0.118	*0.134±0.055*	0.361±0.084
mpg	(7, 392)	0.187±0.014	0.120±0.011	*0.095±0.087*	0.193±0.006	*0.061±0.010*	**0.178**±0.007

[6] Nils M. Kriege, Pierre Louis Giscard, and Richard C. Wilson, "On valid optimal assignment kernels and applications to graph classification," in *Proceedings of Advances in Neural Information Processing Systems*, 2016, pp. 1623–1631.

[7] Matthäus Kleindessner and Ulrike von Luxburg, "Kernel functions based on triplet comparisons," in *Proceedings of Advances in Neural Information Processing Systems*, 2017, pp. 6810–6820.

[8] Xiaolin Huang, Johan A.K. Suykens, Shuning Wang, Joachim Hornegger, and Andreas Maier, "Classification with truncated ℓ_1 distance kernel," *IEEE Transactions on Neural Networks and Learning Systems*, vol. 29, no. 5, pp. 2025 – 2030, 2018.

[9] Gaëlle Loosli, Stéphane Canu, and Soon Ong Cheng, "Learning SVM in Kreĭn spaces," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 38, no. 6, pp. 1204–1216, 2016.

[10] Manik Varma and Bodla Rakesh Babu, "More generality in efficient multiple kernel learning," in *Proceedings of the International Conference on Machine Learning*, 2009, pp. 1065–1072.

[11] Mehmet Gönen and Ethem Alpaydn, "Multiple kernel learning algorithms," *Journal of Machine Learning Research*, vol. 12, pp. 2211–2268, 2011.

[12] Serhat S Bucak, Rong Jin, and Anil K Jain, "Multiple kernel learning for visual object recognition: A review," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 36, no. 7, pp. 1354–1369, 2014.

[13] Pratik Jawanpuria, Jagarlapudi Saketha Nath, and Ganesh Ramakrishnan, "Generalized hierarchical kernel learning," *Journal of Machine Learning Research*, vol. 16, no. 1, pp. 617–652, 2015.

[14] Andrew Gordon Wilson and Ryan Prescott Adams, "Gaussian process kernels for pattern discovery and extrapolation," in *Proceedings of International Conference on Machine Learning*, 2013, pp. 1067–1075.

[15] Andrew Wilson and Hannes Nickisch, "Kernel interpolation for scalable structured gaussian processes (kiss-gp)," in *Proceedings of International Conference on Machine Learning*, 2015, pp. 1775–1784.

[16] Corinna Cortes, Mehryar Mohri, and Afshin Rostamizadeh, "Algorithms for learning kernels based on centered alignment," *Journal of Machine Learning Research*, vol. 13, no. 2, pp. 795–828, 2012.

[17] Aman Sinha and John C Duchi, "Learning kernels with random features," in *Proceedins of Advances in Neural Information Processing Systems*, pp. 1298–1306. 2016.

[18] Gert R. G Lanckriet, Nello Cristianini, Peter Bartlett, Laurent El Ghaoui, and Michael I Jordan, "Learning the kernel matrix with semidefinite programming," *Journal of Machine Learning Research*, vol. 5, no. 1, pp. 27–72, 2004.

[19] Steven C. H. Hoi, Rong Jin, and Michael R. Lyu, "Learning nonparametric kernel matrices from pairwise constraints," in *Proceedings of International Conference on Machine Learning*, 2007, pp. 361–368.

[20] Jinfeng Zhuang, Ivor W Tsang, and Steven C. H Hoi, "A family of simple non-parametric kernel learning algorithms," *Journal of Machine Learning Research*, vol. 12, no. 2, pp. 1313–1347, 2011.

[21] Zhengdong Lu, Prateek Jain, and Inderjit S. Dhillon, "Geometry-aware metric learning," in *Proceedings of International Conference on Machine Learning*, 2009, pp. 673–680.

[22] Jinfeng Zhuang, Ivor W Tsang, and Steven C. H Hoi, "A family of simple non-parametric kernel learning algorithms," *Journal of Machine Learning Research*, vol. 12, no. 2, pp. 1313–1347, 2011.

[23] Xiaojin Zhu, Jaz S. Kandola, Zoubin Ghahramani, and John D. Lafferty, "Nonparametric transforms of graph kernels for semi-supervised learning," in *Proceedings of Advances in Neural Information Processing Systems*, 2005, pp. 1641–1648.

[24] F. Liu, X. Huang, C. Gong, J. Yang, and L. Li, "Nonlinear pairwise layer and its training for kernel learning," in *Proceedings of the AAAI Conference on Artificial Intelligence*, 2018, pp. 3659–3666.

[25] Stephen Boyd and Lieven Vandenberghe, *Convex Optimization*, Cambridge university press, 2004.

[26] Nathan Srebro and Shai Ben-David, "Learning bounds for support vector machines with learned kernels," in *International Conference on Computational Learning Theory*. Springer, 2006, pp. 169–183.

[27] Yoshua Bengio, Jean Francois Paiement, and Pascal Vincent, "Out-of-sample extensions for lle, isomap, mds, eigenmaps, and spectral clustering," in *Proceedings of Advances in Neural Information Processing Systems*, 2004, pp. 177–184.

[28] Michaël Fanuel, Antoine Aspeel, Jean-Charles Delvennes, and Johan A.K. Suykens, "Positive semi-definite embedding for dimensionality reduction and out-of-sample extensions," *arXiv preprint arXiv:1711.07271*, 2017.

[29] Binbin Pan, Wen Sheng Chen, Bo Chen, Chen Xu, and Jianhuang Lai, "Out-of-sample extensions for non-parametric kernel methods," *IEEE Transactions on Neural Networks and Learning Systems*, vol. 28, no. 2, pp. 334–345, 2017.

[30] George P. H. Styan, "Hadamard products and multivariate statistical analysis," *Linear Algebra and Its Applications*, vol. 6, pp. 217–240, 1973.

[31] J. Frédéric Bonnans and Alexander Shapiro, "Optimization problems with perturbations: A guided tour," *SIAM Review*, vol. 40, no. 2, pp. 228–264, 1998.

[32] Yiming Ying, Colin Campbell, and Mark Girolami, "Analysis of SVM with indefinite kernels," in *Proceedings of Advances in Neural Information Processing Systems*, 2009, pp. 2205–2213.

[33] Shiqian Ma, Donald Goldfarb, and Lifeng Chen, "Fixed point and bregman iterative methods for matrix rank minimization," *Mathematical Programming*, vol. 128, no. 1-2, pp. 321–353, 2011.

[34] Nesterov Y., "Smooth minimization of non-smooth functions," *Mathematical programming*, vol. 103, no. 1, pp. 127–152, 2005.

[35] Jongwon Lee, Venkataramanan Balakrishnan, Cheng Kok Koh, and Dan Jiao, "From $\mathcal{O}(k^2N)$ to $\mathcal{O}(N)$: A fast complex-valued eigenvalue solver for large-scale on-chip interconnect analysis," in *Microwave Symposium Digest, 2009. MTT '09. IEEE MTT-S International*, 2009, pp. 181–184.

[36] Subhransu Maji, Alexander C Berg, and Jitendra Malik, "Efficient classification for additive kernel SVMs," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 35, no. 1, pp. 66–77, 2013.

[37] Ali Rahimi and Benjamin Recht, "Random features for large-scale kernel machines," in *Proceedings of Advances in Neural Information Processing Systems*, 2007, pp. 1177–1184.

[38] Shusen Wang, Luo Luo, and Zhihua Zhang, "SPSD matrix approximation vis column selection: theories, algorithms, and extensions," *Journal of Machine Learning Research*, vol. 17, no. 1, pp. 1697–1745, 2016.

[39] Alex J Smola and Bernhard Schölkopf, "Sparse greedy matrix approximation for machine learning," in *Proceedings of International Conference on Machine Learning*, 2000, pp. 911–918.

[40] Cho-Jui Hsieh, Si Si, and Inderjit Dhillon, "A divide-and-conquer solver for kernel support vector machines," in *Proceedings of International Conference on Machine Learning*, 2014, pp. 566–574.

[41] Yuchen Zhang, John Duchi, and Martin Wainwright, "Divide and conquer kernel ridge regression," in *Proceedings of Conference on Learning Theory*, 2013, pp. 592–617.

[42] Si Si, Cho-Jui Hsieh, and Inderjit S Dhillon, "Memory efficient kernel approximation," *Journal of Machine Learning Research*, vol. 18, pp. 1–32, 2017.

[43] Eric V. Strobl and Shyam Visweswaran, "Deep multiple kernel learning," in *Proceedings of International Conference on Machine Learning and Applications*, 2014, pp. 414–417.

[44] Mehmet Gonen, "Bayesian efficient multiple kernel learning," in *Proceedings of the International Conference on Machine Learning*, 2012, pp. 1–8.

[45] Alex Krizhevsky and Geoffrey Hinton, "Learning multiple layers of features from tiny images," *Technical report, University of Toronto*, 2009.

[46] Sergey Ioffe and Christian Szegedy, "Batch normalization: Accelerating deep network training by reducing internal covariate shift," in *Proceedings of International Conference on Machine Learning*, 2015, pp. 448–456.

[47] John C. Platt, "Sequential minimal optimization: A fast algorithm for training support vector machines," in *Advances in Kernel Methods-support Vector Learning*, 1998, pp. 212–223.

[48] Yung-Kyun Noh, Masashi Sugiyama, Kee-Eung Kim, Frank Park, and Daniel D Lee, "Generative local metric learning for kernel regression," in *Proceedings of Advances in Neural Information Processing Systems*, 2017, pp. 2449–2459.

[49] Vladimir Cherkassky, Don Gehring, and Filip Mulier, "Comparison of adaptive methods for function estimation from samples," *IEEE Transactions on Neural Networks*, vol. 7, no. 4, pp. 969–984, 1996.